

基于树搜索的生鲜产品装箱优化研究

摘 要

随着生鲜产品需求的持续增长，冷链物流中涉及多容器、高异质物品的装箱问题已成为提升运输效率、降低物流成本的重要挑战。本文面向典型生鲜运输场景，开展三维装箱优化研究，提出相应建模与算法设计方案。

首先，本文构建了考虑实际约束的强异构物装箱模型，综合考虑了生鲜产品的易损保护、配送与卸货顺序、承重能力、物品支撑稳定性等约束条件，通过合理简化假设，最终建立了最小化容器总数、最大化装载率的多目标装箱模型。

针对模型求解，本文提出了一种基于树搜索与同层装载策略的三维装箱启发式算法。算法整体分为五个步骤：货品装载定序初始化、块生成、同层装载策略、树搜索扩展优化以及方案评估与结果输出。在路径装载定序中，通过先送后装与易损品保护策略，确保卸货效率与货物完好；随后引入了块生成机制，同层装载策略通过相似高度与支撑稳定性的组合，最大化单层装载率；在此基础上，设计了加权装载率与容器数量惩罚项相结合的适应度函数。

在数值实验部分，本文构建三类典型算例，分别为单一箱型、中异质性、高异质性货物，结果表明：箱型异质性对装载率影响显著，装载率由 95% 逐步下降至 76%。分析原因得到，完整层结构、主导箱型支撑能力与边角填补策略是提升装载率的核心要素，并基于此提出四点管理建议：控制箱型种类数量、推进模数化设计、配置主导箱型以及合理配置边角小箱体。随后，通过对支撑面系数进行敏感性分析，实验表明在取值 0.75 时可在稳定性与装载效率之间取得较好平衡。

最后，进行算法有效性分析，与 Gurobi 求解器求解结果进行对比。结果显示：树搜索算法在小规模任务下解的精度与 Gurobi 相近，装载率差距为 2.17%，但求解时间仅为其 17% 左右；在大规模装箱任务中，Gurobi 难以有效求解，而树搜索依然保持稳定性能，装载率维持在 88% 左右，展现出良好的扩展性与实用性。

关键词：三维装箱，树搜索算法，同层装载，适应度函数

Optimization of Fresh Product Packing Based on Tree Search

Abstract

With the continued growth in demand for fresh products, the three-dimensional bin packing problem (3D-BPP) involving multiple containers and highly heterogeneous items has become a key challenge in improving transportation efficiency and reducing logistics costs in cold chain systems. This study focuses on a typical fresh-product logistics scenario and proposes a modeling and algorithmic framework for 3D packing optimization.

First, a multi-objective 3D packing model is established for multi-container loading scenarios with strong item heterogeneity. The model incorporates a range of practical constraints, including protection of perishable goods, delivery and unloading sequence, vehicle weight-bearing capacity, and inter-item stacking stability. After reasonable simplifications, the model aims to minimize the number of containers while maximizing space utilization.

To solve this model, a heuristic algorithm based on tree search and a same-layer packing strategy is proposed. The algorithm consists of five main steps: initialization of loading sequence, block generation, same-layer packing, tree search-based solution expansion, and result evaluation. A “load-later-serve-first” strategy and fragile item protection rule are used to ensure delivery efficiency and item integrity. The block generation mechanism, combined with stacking height similarity and support constraints, improves single-layer packing performance. A fitness function integrating weighted loading rate and container usage penalty is also developed to guide solution selection.

In the numerical experiments, three typical cases—single-type items, medium heterogeneity, and high heterogeneity—are constructed. Results show a significant impact of item heterogeneity on loading rate, with rates declining from 95% to 76%. Analysis reveals that complete layer structure, dominant item support, and corner space filling are critical to packing efficiency. Based on these insights, four managerial suggestions are proposed: limiting item types, promoting modular box design, configuring dominant item types, and allocating small boxes for space filling. Sensitivity analysis on the support area coefficient further shows that a value of 0.75 offers a good trade-off between loading stability and efficiency.

Finally, algorithmic effectiveness is evaluated by comparison with the commercial solver Gurobi. In small-scale tasks, the proposed algorithm achieves comparable solution quality with an average loading rate gap of only 2.17%, while requiring just 17% of the computation time. In large-scale tasks, where Gurobi fails to solve efficiently, the tree search algorithm maintains stable performance with an average loading rate around 88%, demonstrating strong scalability and practical applicability.

Key Words: 3D bin packing, tree search algorithm, same-layer placement, fitness function

目录

摘 要.....	I
Abstract.....	II
第一章 绪论.....	1
1.1 课题背景与意义.....	1
1.1.1 课题研究背景.....	1
1.1.2 课题研究意义.....	2
1.2 课题相关技术的发展概况.....	3
1.2.1 装箱问题研究现状.....	3
1.2.2 三维装箱问题求解算法.....	5
1.3 论文研究内容及技术.....	7
1.3.1 研究内容.....	7
1.3.2 技术路线.....	8
1.4 论文组织结构.....	9
第二章 数学模型.....	11
2.1 生鲜产品三维装箱问题描述.....	11
2.1.1 考虑三维装箱约束的生鲜产品配载特点.....	11
2.1.2 生鲜产品三维装箱问题的建模假设.....	13
2.1.3 装箱问题优化目标.....	13
2.2 三维装箱问题的模型构建.....	14
2.2.1 空间坐标系的定义.....	14
2.2.2 参数和变量说明.....	14
2.2.3 已知参数和决策变量.....	16
2.2.4 目标函数和约束条件.....	17
2.3 本章小结.....	19
第三章 算法设计.....	20
3.1 算法总体框架.....	20
3.1.1 问题定义与输入输出.....	20
3.1.2 重要数据结构定义.....	20
3.1.3 算法整体框架.....	22
3.2 算法模块功能设计.....	24

3.2.1 货品装载定序初始化.....	24
3.2.2 块生成机制设计.....	24
3.2.3 同层装载算法.....	25
3.2.4 树搜索优化策略.....	26
3.2.5 适应度函数设计策略.....	28
3.3 本章小结.....	29
第四章 数值实验及结果分析.....	30
4.1 使用算例说明.....	30
4.2 实验结果及其分析.....	30
4.2.1 单一货品装箱展示.....	31
4.2.2 中等异质性货品装箱展示.....	32
4.2.3 高异质性货品装箱展示.....	35
4.2.4 结果分析.....	38
4.2.5 管理启示.....	38
4.3 支撑面系数 α 敏感性分析.....	39
4.4 与 Gurobi 比较的方法有效性验证.....	40
4.3.1 小规模算例对比分析.....	40
4.3.2 大规模算例对比分析.....	42
4.3.2Gurobi 与树搜索算法对比分析小结.....	43
4.5 本章小结.....	43
第五章总结与展望.....	45
5.1 论文总结.....	45
5.2 研究展望.....	45
参考文献.....	47
附录.....	52
致谢.....	70

第一章 绪论

1.1 课题背景与意义

1.1.1 课题研究背景

在物流行业整体蓬勃发展的大背景下，生鲜物流作为其日益重要的细分领域，受到了越来越多的关注。生鲜物流，又被称为生鲜冷藏物流或冷藏物流，意思是在严格控制的低温环境下，对生鲜产品加工、储运和销售的物流体系。随着居民消费水平的提高和消费结构的升级，生鲜产品的需求量持续增长，对生鲜物流的效率和品质提出了更高要求。根据《中国冷链物流发展报告》显示，2023 年我国冷链物流市场规模已突破 5000 亿元，预计未来几年仍将保持高速增长。这说明了生鲜物流在我国物流行业中占据着越来越重要的战略地位。与传统的海运集装箱运输不同，生鲜物流多数采用独立的冷链运输方式，对时效性和产品价值的保障提出了更为严苛的要求。

在政策层面，国家高度重视物流成本控制与智能装载技术的推广。2024 年 1 月 27 日，中共中央办公厅、国务院办公厅联合印发的《有效降低全社会物流成本行动方案》中明确提出，要加快推进现代物流体系建设，将装箱列为关键支撑环节之一，强调通过自动化、标准化手段降低装载成本、提升运输效率与服务水平。

生鲜产品具有其特殊性。就其生鲜产品本身而言，具有鲜活性、区域性、季节性、环境敏感性、易损性等特性^[1]。例如，苹果在运输过程中会持续进行呼吸作用，释放生物热；生鲜产品对冷风直吹较为敏感，容易发生冻伤。考虑到生鲜产品的易损性，保证装载过程中的合理性，制定切实可行的方案尤其重要，易遭受挤压的货品应该放在最后装载，且摆放在其他物品上层，以避免在运输中遭到挤压，这样的装载原则利于减少农产品在配送过程中的损耗，确保生鲜产品的货品质量与完整性。其次，在装配中心进行提前装载时，应当遵从“先送后装”的顺序原则，以确保配送卸货任务的时效性和效率。

综上所述，对生鲜产品冷藏集装箱的三维装箱优化问题进行研究具有重要的理论和实际意义。在单次运输、单个集装箱内尽可能存储、排布更多质量、不同种类的生鲜产品，提高货物的装载率是提高运输经济效益的有效手段^[3]。

然而，现有的生鲜物流装箱问题大多采用经验式决策，企业端也对货物装箱效率提出了更高要求。传统的人工装载方式在应对形态差异大、规则不一的生鲜货物时效率低下，装载率不高。特别是在电商、生鲜零售、制造业等领域的快速发展背景下，企业急需依靠智能装箱算法来替代经验式操作，提升冷链运输过程中的装载效率与稳定性。

因此，开展针对性的生鲜产品三维装箱优化研究，应用运筹学、计算机技术等科学手段，得到有效的三维装箱方案，是物流企业降低运输损耗、提高装载率、提升运输经济效益的有效途径。

1.1.2 课题研究意义

本课题的研究意义体现在理论层面与实际应用层面。

在理论层面，生鲜产品的运输装载过程本质上属于典型的三维装箱问题，本课题的研究弥补了生鲜产品在三维装箱研究领域的空白，具有重要的理论价值。三维装箱问题作为经典的 NP-hard 问题^[4]，其复杂性和挑战性一直吸引着众多学者关注。但是现有的研究大多集中于工业产品等标准化货物的装箱优化，对于生鲜产品这一类的装载约束复杂的物品关注相对较少。现有的关于生鲜产品的研究大多数以车辆路径优化问题、选址问题为主，缺乏针对生鲜产品这类异质性强、限制条件复杂的装载场景的装箱建模与求解工作。为填补这一空白，本文结合实际装载需求，构建了多容器三维装箱数学模型，系统引入易损保护、卸货顺序、承重能力与支撑稳定性等多种约束条件，并基于树搜索思想设计了同层装载策略与适应度函数结合的启发式求解算法，在传统三维装箱问题的建模与算法设计方面进行了有针对性的拓展，丰富了该类非标准化物品装箱优化的理论体系。

在实践层面，科学的装箱方案能够提高空间利用率、降低物流成本，提高运输效率^[5]。本文聚焦生鲜产品装箱过程中的典型需求，提出的模型与算法可以提升空间利用率、降低损伤、保障装卸顺序的可行性，从而在多点配送、批量分拣等应用中具备良好的适应性。研究成果不仅适用于果蔬、肉禽、水产等生鲜类商

品的配载，也可推广至药品、日化品、轻工品等对摆放方向和稳定性有特殊要求的商品，为相关企业优化运输方案、提升运营效率提供有力支撑。

1.2 课题相关技术的发展概况

下面，本文将结合国内外文献研究进展综述与本文相关的装箱问题概况。

1.2.1 装箱问题研究现状

装箱问题（Bin Packing Problem, BPP）作为一个经典的组合优化问题，在现实生活中有着广泛的应用。它的目标是将一组物品放入一个或多个容器中，使得容器的装载率最大化。根据物品和容器的尺寸，装箱问题可以分为一维、二维和三维装箱问题。

（1）一维装箱问题

一维装箱问题（One-Dimensional Bin Packing Problem, 1D-BPP）作为装箱问题中最基础的形式，其核心在于如何将一组长度或重量不等的物品有效地装入容量相同的线性容器中，以最小化容器的使用数量或最大化容器的利用率。本质上，1D-BPP 可以被建模为一个 0-1 整数规划问题，同时也被证明属于 NP 完全问题，这意味着随着问题规模的增大，求解难度将呈指数级上升。例如，经典的背包问题、车辆调度问题以及工厂排程问题，在一定程度上都可以抽象为 1D-BPP 进行求解^[6]。

1D-BPP 的研究可以追溯到上个世纪六七十年代。1971 年，Ullman J. D.^[7]提出了首次适应算法（First Fit, FF），遵循“先到先得”的原则，也就是按照容器打开的先后顺序，将物品放入第一个能够容纳它的容器中。尽管实现方法简单，但在一些情景下 FF 算法难以达到理想的装载效果。1979 年，Johnson 等人^[8]提出了下次适应算法（Next Fit, NF），只考虑当前打开的容器，一旦物品无法放入，则立即打开下一个新容器，但这个算法容易导致较高的资源浪费^[9]。除了 FF 和 NF 算法，还有最佳适应算法（Best Fit, BF）、降序首次适应算法（First Fit Decreasing, FFD）、降序最佳适应算法（Best Fit Decreasing, BFD）等算法。其中，FFD 和 BFD 作为离线启发式算法，在实际应用中表现出较好的性能。1984 年，Coffman 等人^[10]对 1D-BPP 的相关问题和算法进行了全面系统的总结，为后续研究奠定了坚实的基础。1990 年，Sweeney 等人^[11]针对一维切割库存问题中主卷和客户订

单具有多种质量等级的情况，提出了一种基于影子价格的两阶段启发式算法，该算法能够有效处理不同规模和复杂性的问题，并适应生产质量和需求的变化近年来，1D-BPP 的研究重点逐渐转向多种装箱算法的融合，并结合实际应用场景中的约束条件，以期获得更好的装载效果^[12]。2021 年，巩梨等人^[13]针对传统一维装箱问题中容器容量均一的局限，构建了多目标优化模型，并提出随机机会约束规划方法，有效解决了实际运输中容器费用随机性的问题。

（2）二维装箱问题

二维装箱问题（Two-Dimensional Bin Packing Problem, 2D-BPP）相较于一维装箱问题，其复杂性显著提升。该问题不仅需要考虑物品的长度，还需兼顾其宽度，目标是在有限的矩形容器内，合理安排一组尺寸各异的矩形物品，以期达到容器利用率的最大化或容器使用数量的最小化。2D-BPP 在实际生活中有着广泛的应用场景，如服装行业的裁剪排版、玻璃等易损物品的运输装载、以及广告牌的优化设计等，都可抽象为 2D-BPP 进行求解。

针对 2D-BPP，Jakobs^[14]等人提出了基于左下角算法的启发式算法来解决正交矩形 packing 问题，提高装箱效率。Hopper^[15]等人则专注于研究不含“断头台”截断约束的 ORSP 问题，他们提出了基于 Bottom-Left 的 Bottom-Left-Fill 算法，结合遗传算法和 Naïve Evolution 算法等元启发式算法，形成了较为高效的混合启发式算法。这些混合算法充分融合了不同算法的优势，在解的质量和求解效率上都取得了显著的提升。Martell^[16]等人则从数学规划的角度出发，提出了 ORSP 问题的线性松弛模型，并基于此模型导出了三种新的下界。Kun Peng^[17]研究了带冲突约束的 2D-BPP，提出 IMA 启发式算法并结合局部搜索，有效平衡冲突惩罚和容器用量。实验表明该算法稳定有效。Lodi^[18]关注 guillotine 切割约束的 2D-BPP，提出局部枚举算法，通过限制切割次数 K 求解。Laurent 等人^[19]基于杂交算法提出新粒子群算法，用于资源分配。该算法对类似约束的 2D-BPP 问题有借鉴意义，实验显示效果良好。

（3）三维装箱问题

三维装箱问题（3D Bin Packing Problem, 3D-BPP）作为经典的组合优化问题，因其 NP-hard 的特性，一直是研究的热点。该问题需同时考虑物品的长、宽、高三个维度，三维装箱问题的目标是将一组尺寸不等的长方体物品放入尺寸相同的

长方体容器中，使得使用的容器数量最少。不存在多项式时间复杂度的最优求解算法，因此针对三维装箱算法的研究集中在启发式算法和近似算法^[20]。

早在 1991 年，Scheithauer^[21]率先提出了 3D-BPP 的近似算法。随后在 2002 年，Lodi 和 Martello^[22]提出了禁忌搜索算法，该算法通过允许接受较差解，有效避免了局部最优。2009 年，Faroehe1 和 Zachariasen^[23]提出的引导式局部搜索，则通过贪婪算法获取上界，迭代减少容器使用量，直至满足终止条件。该算法不仅适用于 3D-BPP，也适用于 2D-BPP。2008 年，Crainic^[24]在研究中引入了极值点的概念，并提出了一种基于该概念的启发式算法。

近年来，研究者们关注算法改进。2010 年，Wu 等^[25]针对三维装箱问题的一种变形进行了研究，即在底面长宽固定的情况下，求解装入一定货物所需的最小高度，采用混合整数规划，通过设定货物摆放的约束条件来求解问题。2012 年，Kang 等^[26]通过制定一种包装策略，以最小化包装过程中产生的剩余空间为目标，并结合遗传算法进行求解，提高了装载率。2018 年，吴自立^[27]在其关于箱型设计及装箱问题的算法研究中，对装箱问题进行了系统分类，并详细分析了关键点法、空间分割法和分层法等经典算法。在此基础上，结合所分析的算法的优点，提出了点面结合的复空间求解方法，提高了求解效率。Gzara^[28]于 2020 年研究带托盘装载问题，提出基于层的列生成方法，采用二阶锥规划和图形表示处理约束。Krzysztof^[29]提出基于时间窗口和压缩网络流的 MILP 模型，以压缩网络流的形式表示物品在物品中的打包过程，提高了模型的求解效率和灵活性。Tian 等^[30]提出一种基于深度强化学习的序列到序列策略网络（S2SDRL）方法，通过优化物品包装顺序，提高多车辆协同装箱问题的车辆装载率。

1.2.2 三维装箱问题求解算法

三维装箱问题是经典的 NP-hard 问题，也就是说随着问题规模的增大，求解时间呈指数级增长，求解难度较高。本章将对其经典求解算法进行系统性综述。

（1）精确算法

精确算法目的是找到模型的最优解，一般适用于小规模问题的求解。Valério 等^[31]针对一维装箱问题建立了整数规划模型，并通过线性松弛获得下界，结合分支定界算法进行求解。Muritiba 等^[32]考虑了带冲突约束的装箱问题，部分物品对

不能共存于同一容器，基于分支定界算法，提出了新的上下界计算方法。Vanderbeck 等人^[33]通过利用列生成算法，精确求解装箱问题。

近年来，有一些研究将 3D-BPP 建模为数学规划问题，并用求解器求解。Chen^[34]和 Junqueira 等^[35]把 3D-BPP 建模为混合整数线性规划（MILP）问题，用求解器求解精确解。Hifi 等^[36]建立 MILP 模型，以最小化容器使用量为目标，并通过引入有效不等式改进模型的松弛下界。总而言之，精确算法能够保证得到最优解，但受限于计算复杂度，仅适用于求解小规模 3D-BPP。

（2）启发式算法

启发式算法是一种基于经验或直观的算法，旨在快速找到一个较好的可行解，不一定是最优解。该算法适用于大规模问题，常见的启发式算法包括构造性启发式算法、局部搜索启发式算法等。2013 年，Ceschia 等^[37]使用了一种基本构建方法，并基于 TS 和模拟退火开发了两种元式算法，以解决 CLP 问题。2002 年，Lodi 等^[38]提出了一个层建筑物的启发式，把一部分盒子的子集包装到最小数量的垃圾箱中，并使用 TS 方法来通过减少层总数来优化初始解决方案。Olsson 等^[39]使用了一种贪婪的建设性启发式，先将盒子包装成堆栈，再使用加权评分函数评估潜在的位置。Dereli 等^[40]开发了一种多目标模型获得 SKP 的有效填料模式，并提出了一种带有墙壁建筑物启发式的 SA 算法来解决该模型。Sheng 等^[41]提出了一种基于墙壁建筑物的启发式方法解决 CLP，并考虑了实际的约束，如盒子方向和垂直稳定性。。

（3）树搜索算法

在三维装箱问题（3D-BPP）中，树搜索算法通过迭代地将物品装入容器来构建解决方案。2002 年，Pisinger^[42]提出了一种基于墙体构建策略的启发式方法，结合树搜索和排名规则来确定物品的放置，将容器分成垂直层，再细分为条带，并在树搜索的每个节点上，根据排名规则选择墙体深度和条带宽度优化填充条带。2011 年，Tian 和 Sawaragi 等人^[43]提出了一种基于块构建的树搜索算法解决单一容器装载问题。2014 年，Araya 等人^[44]引入了树搜索算法，使用贪婪函数评估搜索树中的状态，该算法基于物品体积对块进行排名，并使用曼哈顿距离选择自由空间。2017 年，Guerrero 等人^[45]改进了 Araya 的算法，引入评估函数来最大化装载率。2020 年，Araya 等人^[46]调整了先前的贪婪树搜索算法，以处理具有双重目

标的单一 CLP 变体，除了最大化体积利用率外还考虑了最大化装载物品的总利润（如重量或优先级）。总而言之，树搜索算在解决 3D-BPP 发挥着重要作用，通过不同的构建策略、评估函数和搜索技术，对提高装箱效率有着重要作用。

（4）深度强化学习算法

深度强化学习能够直接从原始数据中学习，允许模型直接从物品和物品信息中学习装箱策略，无需人工设计规则，具有更强的泛化能力。近年来，部分学者开始尝试利用深度强化学习方法求解装箱问题。例如，Liu 等人^[47]则研究了物品形状数量有限的三维装箱问题，将相同物品组合成物品块，再利用深度强化学习选择物品块并确定包装位置，最终达到 88.39% 的装载率。Jiang 等人^[48]为了提高算法效率，分解为序列、方向和位置三个子任务分别训练，成功解决了更大规模的装箱问题。

（5）近似算法

近似算法是一种能够在多项式时间内找到近似最优解的算法，通过牺牲一定的解的质量，以换取更快的求解速度。然而在装箱问题中，经典的近似算法如 NF 和 BF 性能有限。为提升性能，Johnson 等^[49]提出先将物品降序排序，再应用 NF 或 BF 算法。Kang 等^[50]针对物品尺寸可变的装箱问题，以最小化物品总成本为目标，基于降序首次适应等算法求解，获得了较好的装箱方案。

1.3 论文研究内容及技术

1.3.1 研究内容

装箱优化技术是物流运作与管理中一项十分重要技术，课题研究生鲜产品装箱优化模型与算法，有良好借鉴和参考作用。三维装箱问题（3D-BPP）是一类难以解决的组合优化问题，在过去的几十年里，已经有许多学者对其进行了研究。装箱问题有很多变体，但 3D-BPP 是装箱领域中最困难的问题之一。生鲜产品装箱优化问题作为三维装箱问题的一种扩展，更加符合现代社会生鲜装载生产的需要。

生鲜产品装箱优化问题是一个 NP 难题，其中装箱顺序和物品摆放所产生的装载率和产品质量对生鲜装载产生了严重影响。根据调查，物流运输占全球能源消耗总量的相当一部分。随着食品安全和节能减排意识的增强，生鲜产品装箱优化问题引起了国内外学者的关注。考虑到问题的高度复杂性，大多数研究者倾向于采用智能优化算法来解决此类问题，包括遗传算法、模拟退火算法、粒子群优

化、蚁群优化、树搜索算法等，其中树搜索算法是一种高效的启发式搜索算法，它同时是解决组合优化问题中最重要的算法之一。以上这些优化算法已被广泛应用于装箱优化领域。在算法的选择上要综合考虑各算法在求解多目标三维装箱问题上的优势，结合所建立的模型应用更具有针对性的算法，来实现对实际问题的处理与优化。

1.3.2 技术路线

图 1 呈现了本文的技术路线图，包括问题提出、建立模型、设计算法框架以及算法有效性验证。针对生鲜产品装箱优化算法设计，本文构建了生鲜产品装箱优化的搜索空间模型，并利用树搜索算法训练搜索过程进行装箱，实现了对该问题的求解。

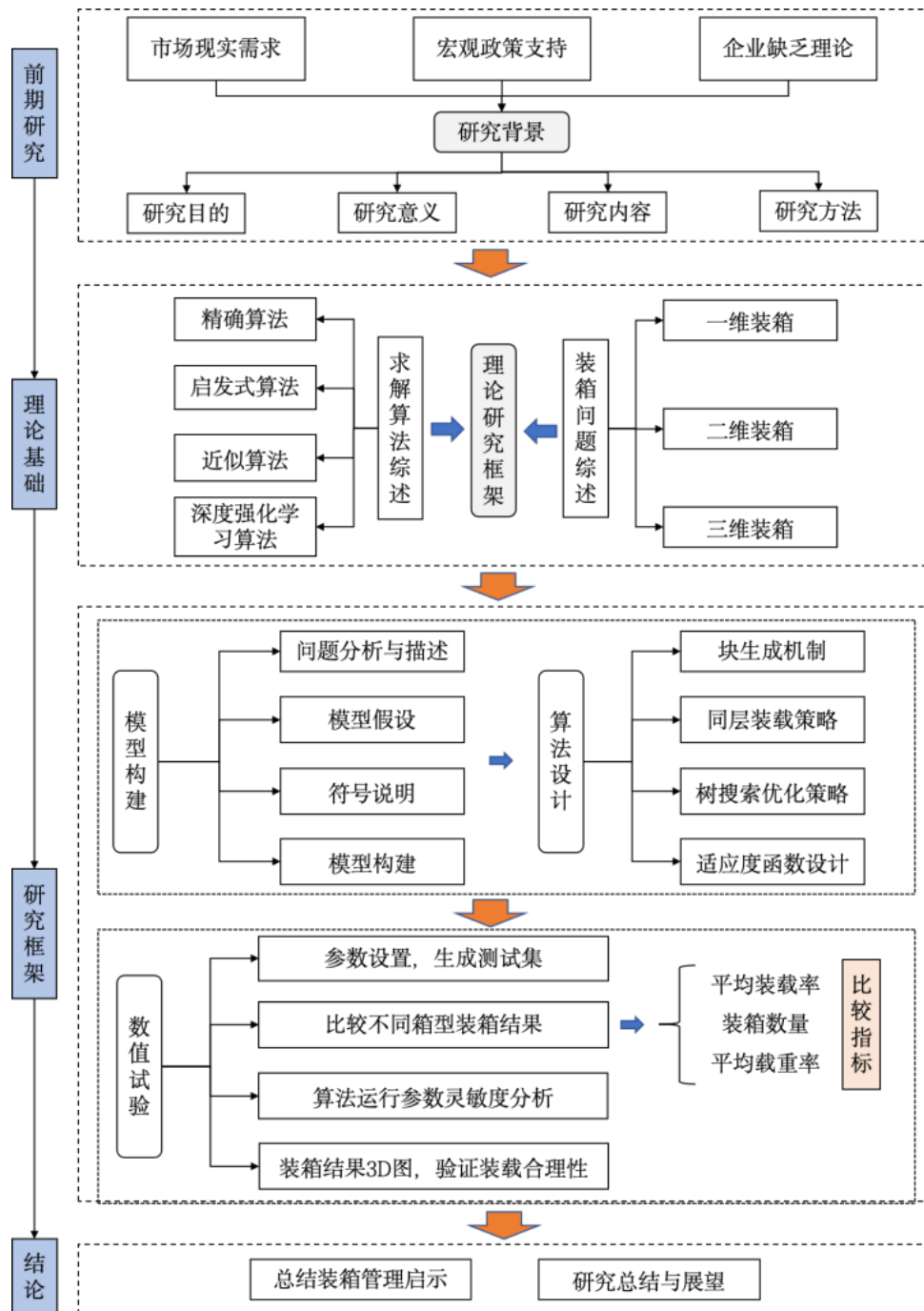


图 1-1 技术路线图

1.4 论文组织结构

本论文共分为五章，结构安排如下：

第一章绪论，介绍了课题研究背景与意义，指出生鲜装载中生鲜产品三维装箱问题在当前物流体系中的重要性与现实挑战，阐述了其在提升装载率、保障食品安全、节能降耗等方面的重要价值。随后综述了不同维度装箱问题的研究现状，

重点回顾了三维装箱问题中的经典求解算法与最新发展，并介绍了树搜索算法在该领域的应用前景，最后明确了本课题的研究内容与技术路线。

第二章，围绕生鲜产品三维装箱优化问题，建立了数学建模基础。在分析实际场景约束的基础上，提出了建模假设，定义了坐标系、已知参数与决策变量，并构建了包括不重叠、堆叠支撑、容器边界、装载方向等在内的完整约束系统。同时，结合物流实际需求，设置了双目标优化函数——最小化容器使用数量与最大化装载率。通过明晰的模型构建为后续算法设计与实验验证奠定理论基础。

第三章，算法设计与实现，详细介绍了本文提出的树搜索与同层装载相结合的求解算法。首先考虑到装载顺序和物品易损性，对货品放入的顺序做重新排列，设计了基于块结构的物品组合机制，并提出了同层装载策略以生成高质量初始解。随后构建了基于启发式评估与剪枝机制的树搜索框架，结合状态扩展策略与接触面积评价函数 VCS 进行状态选择。为实现算法收敛与方案评估，设计了适应度评价函数 $fitness$ ，融合装载率、容器数量等多个指标，对搜索过程进行动态引导与筛选。

第四章，数值实验与结果分析，通过构建三类典型算例，分别为单一箱型、中异质性、高异质性，验证箱型异质性对装载率的显著影响，并对装箱结果可视化，验证装载方案的合理性。接着进行敏感性分析，寻找最优参数，为企业管理者对相关参数的决策提供依据。最后，对算法进行有效性分析。

第五章，对本文的工作进行了总结。

第二章 数学模型

2.1 生鲜产品三维装箱问题描述

本章旨在从实际物流装箱任务出发，提出模型假设，定义数学变量，给出数学模型，并配合图示清晰展现生鲜装箱问题中的结构、旋转与放置逻辑，为后续算法设计与求解提供建模基础。装箱问题如图 2-1 所示。

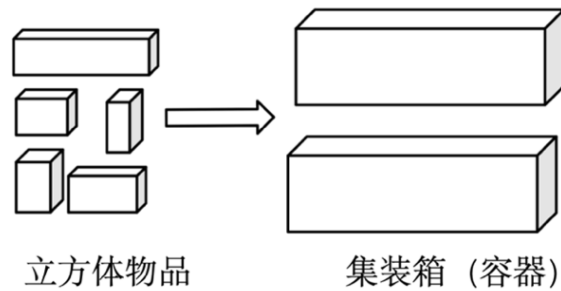


图 2-1 装箱问题示意图

本课题其本质为异质化物品的容器装箱优化问题，其形式化的定义描述如下：给定一组异质化的物品集合 $I = \{1, 2, \dots, n\}$ 和一组箱子集合 $J = \{1, 2, \dots, m\}$ ，每个物品 $i \in I$ 具有固定的长、宽、高 (l_i, w_i, h_i) 和重量 a_i ，每个箱子 $j \in J$ 具有固定的长、宽、高 (L_j, W_j, H_j) 和承重上限 A_j 。目标是在满足各种装载约束的情况下，将所有物品装载到尽可能少的箱子中，并且最大化装载率。

2.1.1 考虑三维装箱约束的生鲜产品配载特点

生鲜装载中的三维装箱问题可以描述为：根据容器尺寸、容器容积以及载重限制，将不同体积、不同重量、不同存放要求的生鲜产品合理地摆放到冷链容器中，以实现最高的载重利用率与装载率，从而保障生鲜产品在配送过程中的新鲜度和质量稳定性。本部分主要考虑生鲜产品在装载过程中的三维尺寸、摆放方向、易损性、支撑面、同温区管理以及先进先出等约束，旨在实现生鲜产品的高效装载，确保装载的稳定性以及配送过程中的安全性，减少生鲜产品在运输中的损耗，满足实际配送的需求。

（1） 生鲜产品的三维外形特点分析

生鲜产品的包装多种多样，以标准化的长方体、塑料周转箱、纸质包装箱为主，也存在部分不规则包装形式。通常来说，不同种类的生鲜产品，如水果、蔬菜、肉类等，其包装尺寸和重量都存在明显差异。针对这些特性，生鲜产品的配载不仅需要保证不超出容器的容积和载重限制，还需要合理规划不同品类的堆叠和放置，避免相互挤压和重叠破坏，以更高效、更安全地利用空间容量。此外，为确保生鲜产品的装载稳定性和运输安全性，在相互接触的生鲜箱体之间，需要考虑足够的支撑面积，以减少因振动引发的倾斜与压损。

（2） 生鲜产品的不可倒置性

生鲜产品中，部分水果和蔬菜，例如西红柿、草莓、葡萄等，在运输过程中具有极高的易损性，对外部压力与翻转非常敏感。因此，在装载过程中，这些产品的高度方向必须与容器高度方向平行，不能倒置或倾斜摆放，避免因重力作用和容器震动导致挤压与碰撞。

（3） 生鲜产品易损性对摆放要求的影响

本文研究的生鲜产品易损性主要是根据其包装方式、物品质量、生鲜特性决定的。生鲜产品主要分为高端水果和普通果蔬两大类，其中高端水果（如进口樱桃、蓝莓等）通常采用多层缓冲包装，一般视为易损品，且质量较小，一般放置于上层。

相对而言，普通的蔬菜和瓜果（如西瓜、白菜等）一般质量较大，因此，在装载时，这类产品应尽量靠近底层摆放，借助重力稳定减少震动带来的破损风险，这类包装方式下的生鲜产品可定义为非易损品。

（4） 装箱过程中的先进后出原则

“先进后出”（First In Last Out, FILO）原则要求需要按照配送路径的服务顺序的逆序进行生鲜产品的堆放。当车辆到达某一客户点卸货时，无需搬动或重新调整其他未配送的生鲜产品，即可顺利完成卸货操作，从而减少配送人员的搬运时间和劳动强度，避免因反复挪动导致的生鲜产品损伤。也就是说，装载时将最先需要配送的生鲜产品放在容器的外层，而后续配送的产品依次向内摆放，最晚送达的产品则位于容器的最内层。这样，当车辆按既定路线行驶时，每到达一个

客户点，只需依次卸货，无需移动其他未配送的产品，也避免了相互堆压的情况，也确保了配送卸货任务的时效性和效率。

2.1.2 生鲜产品三维装箱问题的建模假设

本文根据实际应用场景做出以下假设：

1. 每个物品必须且只能被装载在一个容器中；
2. 每个容器内的物品摆放必须满足不重叠和不出界的要求；
3. 物品可以在底面上任意正交旋转 90 度以适应空间布局，但不可上下旋转；
4. 物品可堆叠，但上层物品必须有足够支撑面，以确保运输过程中的稳定性和防止倾斜倒塌
5. 每个容器存在承重与体积上限，不得超限使用；
6. 所有参数（尺寸、重量、装载条件）在优化前已知，问题为静态一次性装箱。
7. 易损物品应该尽量摆放在上层位置，以避免在运输过程中受到挤压和损坏；
8. 先进后出约束，在装载过程中，需要按照配送路径的服务顺序的逆序进行生鲜产品的堆放。

2.1.3 装箱问题优化目标

本研究面向冷链运输场景中的三维异构货物装箱问题，旨在实现货物的高效装载与运输稳定性的协同优化。考虑到实际应用中箱型多样、货物体积不一、运输空间有限，本文将问题建模为一个多目标优化问题，目标包括：最小化所需容器数量、最大化装载率。

需要注意的是，本文所采用的装载率定义与传统定义方式存在差异。常规装载率通常计算为所有已装载物品总体积与使用容器最大体积之和的比值，即以容器长、宽、高的乘积作为分母，无论容器实际装载高度如何，未被利用的顶部空间也被一并计入。由于本文的设定是物品数量一定，但是容器数量不限，当所用容器数量一致时，这种方式在评估装载率时无法真实反映装箱结构的紧凑程度。为更加准确地衡量实际的空间利用效果，本文重新定义了装载率指标。新定义中，分母不再取容器的最大体积，而是将每个容器的长、宽与其内部最高物品顶部位置相乘，反映该容器实际被占用的空间高度。再将所有容器的“有效占用体积”累加，构成总分母。这种定义方式能够更真实地反映被利用的空间体积，有效避免顶部空置对装载率评估的干扰，从而更科学地衡量装载策略的优劣。

2.2 三维装箱问题的模型构建

根据前述算法逻辑与建模过程，为构建适用于场景的三维装箱优化模型，本文对本章节中涉及的变量、符号、参数及核心数据结构作如下统一定义。

2.2.1 空间坐标系的定义

如下图 2-2 所示，在集装箱内建立三维笛卡尔坐标系，托盘 $\text{back} \leftrightarrow \text{front}$ 方向上建立 X 轴， $\text{left} \leftrightarrow \text{right}$ 方向是建立 Y 轴， $\text{bottom} \leftrightarrow \text{top}$ 建立 Z 轴，X 轴、Y 轴、Z 轴的交叉点即托盘的左后下角为原点，坐标为(0,0,0)摆放入托盘的货物的坐标可以根据货物的左后下角和原点的相对位置来计算表示。

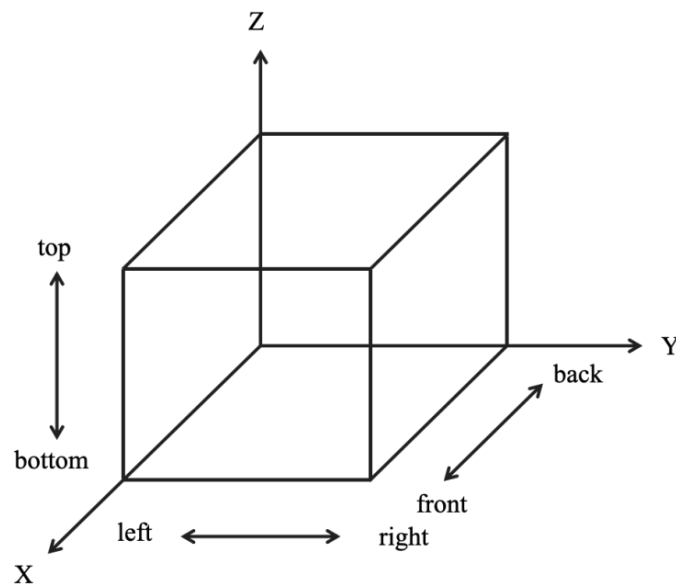


图 2-2 三维笛卡尔坐标系

2.2.2 参数和变量说明

为了更清晰地刻画本算法中的输入输出结构、控制参数与数据表达方式，本文对优化模型中涉及的关键变量、控制参数、评价指标与核心结构体进行了系统整理，并按照其功能划分为五类：容器基础属性、装载约束参数、状态描述变量与核心数据结构。

表 2.1 中定义了集装箱的基本尺寸参数，用于约束三维装载空间，所有值均为装箱过程中的边界限制条件。

表 2.1 容器基本参数属性

参数名	数据类型	解释	使用场景
max_load_length	float	容器的最大可用长度	约束 X 轴范围
max_load_width	float	容器的最大可用宽度	约束 Y 轴范围
max_load_height	float	容器的最大可用高度	约束 Z 轴范围
min_box_gap	float	物品之间的最小间隙	保证装卸操作

表 2.2 设置了装载过程中的约束条件参数，主要影响装载稳定性与装载层形成的可行性。

表 2.2 装载约束参数属性

参数名	默认值	解释	影响因素
min_support_area_rate	0.75	最小支撑面积比例	装载稳定性
max_projected_area_rate	0.85	最大投影面积比例	堆叠稳定性
height_diff_in_one_layer	15	同层高度差异限制	层装载平整度
load_rate_same_layer	0.85	同层最小装载率	层利用效率
load_rate_relax	0.75	装载率松弛阈值	初始解生成

表 2.3 为在装载状态管理中涉及的主要变量，用于记录当前状态并在装载决策中动态更新。

表 2.3 状态描述变量属性

变量名	数据类型	属性说明	更新机制	使用场景
location	Location 类	三维坐标 (x,y,z)	放置时确定	记录位置
shape	Shape 类	三维尺寸 (l,w,h)	初始化固定	描述形状
volume	float	体积	动态累计	装载率计算
load_rate	float	装载率	实时更新	效果评估
fitness	float	适应度值	状态扩展更新	解质量评价
vcs	float	接触面积评分	决策时计算	选择位置

表 2.4 汇总了本算法中的核心数据结构属性，每个类封装了装载问题中的关键对象，用于支持算法模块化运行。

表 2.4 核心数据结构属性

类名	主要属性	核心方法	功能说明	使用场景
Block	boxes, location, volume	combine_blocks()	块管理	块生成与装载
Space	location, shape, anchor	set_anchor()	空间管理	可用空间维护
PackingSet	loaded_blocks, spaces	evaluate()	集合管理	单容器装载
PackingState	packing_sets, fitness	calculate_fitness()	状态管理	整体评估

2.2.3 已知参数和决策变量

已知参数	
n	物品数量
m	容器数量
l_i, w_i, h_i	物品 i 的长度
a_i	物品 i 的重量
L_j, W_j, H_j	容器 j 的长度、宽度、高度
A_j	容器 j 的承重上限
M	足够大的常数，用于约束线性化
T_i	物品 i 的温度需求
决策变量	
p_{ij}	若物品 i 被装入容器 j ，则为 1，否则为 0。
u_j	若容器 j 被使用，则为 1，否则为 0。
x_{ik}^+	若物品 i 在容器中位于物品 k 的左侧，则为 1；否则为 0。
x_{ik}^-	若物品 i 在容器中位于物品 k 的右侧，则为 1；否则为 0。
y_{ik}^+	若物品 i 在容器中位于物品 k 的后方（沿 y 轴更大），则为 1；否则为 0。

y_{ik}^-	若物品 i 在容器中位于物品 k 的前方（沿 y 轴更小），则为 1；否则为 0。
z_{ik}^+	若物品 i 在容器中位于物品 k 的下方（沿 z 轴更小），则为 1；否则为 0。
z_{ik}^-	若物品 i 在容器中位于物品 k 的上方（沿 z 轴更大），则为 1；否则为 0。
l_i^x	若物品 i 的长度方向与 x 轴平行，则为 1；否则为 0。
l_i^y	若物品 i 的长度方向与 y 轴平行，则为 1；否则为 0。
w_i^x	若物品 i 的宽度方向与 x 轴平行，则为 1；否则为 0。
w_i^y	若物品 i 的宽度方向与 y 轴平行，则为 1；否则为 0。
f_i	表示物品 i 是否是易损品，若是则为 1，否则为 0。
x_i, y_i, z_i	物品 i 在容器中的左-底-后角坐标，连续非负变量。

2.2.4 目标函数和约束条件

目标函数：

本模型考虑二重目标，分别为：最大化装载率，最小化所使用的容器数量。

$$f_1 = \max \frac{\sum_{i=1}^n \sum_{j=1}^m (p_{ij} l_i w_i h_i)}{\sum_{j=1}^m (L_j W_j H_j^{load})} \quad (2.1)$$

$$f_2 = \min \sum_{j \in J} u_j \quad (2.2)$$

其中， $H_j^{load} \geq z_k + h_k + M(1 - z_{ik}^-), \forall i \in I$

此外，还需要考虑易损品保护约束，易损物品可放在易损物品上面，易损物品可放在非易损物品上面，但非易损品不能放在易损品上面。定义易损品为 $f_i = 1$ ，非易损品为 $f_i = 5$ ，只需要让易损品尽可能放上方即可，此约束可转化为目标函数 f_3 。

$$f_3 = \min \sum_{j \in J} \left| \frac{\sum_i f_i p_{ij} (z_i + \frac{1}{2} h_i)}{\sum_i f_i p_{ij}} \right| \quad (2.3)$$

约束条件：

约束(1)：物品只能放入已使用的容器。

$$p_{ij} \leq u_j, \forall i \in I, \forall j \in J \quad (2.4)$$

约束(2)：每个物品只能分配到一个容器中。

$$\sum_{j \in J} p_{ij} = 1, \forall i \in I \quad (2.5)$$

约束(3)-(5)：装箱物品不得超出箱体边界。

$$x_i + l_i l_i^x + w_i w_i^x \leq L_j u_j + (1 - p_{ij})M, \forall i \in I, \forall j \in J \quad (2.6)$$

$$y_i + l_i l_i^y + w_i w_i^y \leq W_j u_j + (1 - p_{ij})M, \forall i \in I, \forall j \in J \quad (2.7)$$

$$z_i + h_i \leq H_j u_j + (1 - p_{ij})M, \forall i \in I, \forall j \in J \quad (2.8)$$

约束(6)-(11)：避免物品之间发生空间重叠。对于任意两个物品 i 和 k ，如果它们被分配到同一个容器中，则在 x 、 y 、 z 三个维度中至少存在一个方向不重叠。

$$x_i + l_i l_i^x + w_i w_i^x \leq x_k + (1 - x_{ik}^+)M, \forall i, k \in I, i < k \quad (2.9)$$

$$x_k + l_k l_k^x + w_k w_k^x \leq x_i + (1 - x_{ik}^-)M, \forall i, k \in I, i < k \quad (2.10)$$

$$y_i + l_i l_i^y + w_i w_i^y \leq y_k + (1 - y_{ik}^+)M, \forall i, k \in I, i < k \quad (2.11)$$

$$y_k + l_k l_k^y + w_k w_k^y \leq y_i + (1 - y_{ik}^-)M, \forall i, k \in I, i < k \quad (2.12)$$

$$z_i + h_i \leq z_k + (1 - z_{ik}^+)M, \forall i, k \in I, i < k \quad (2.13)$$

$$z_k + h_k \leq z_i + (1 - z_{ik}^-)M, \forall i, k \in I, i < k \quad (2.14)$$

约束(12)：同箱内物品必须存在至少一个方向上的相对位置。

$$x_{ik}^+ + x_{ik}^- + y_{ik}^+ + y_{ik}^- + z_{ik}^+ + z_{ik}^- \geq p_{ij} + p_{kj} - 1, \forall i, k \in I, i < k, \forall j \in J \quad (2.15)$$

约束(13)-(16)：每个物品只能采用两种正交摆放方式，确保长度宽度分别对齐 x 或 y 轴。

$$l_i^x + l_i^y = 1, \forall i \in I \quad (2.16)$$

$$l_i^x + w_i^x = 1, \forall i \in I \quad (2.17)$$

$$w_i^x + w_i^y = 1, \forall i \in I \quad (2.18)$$

$$l_i^y + w_i^y = 1, \forall i \in I \quad (2.19)$$

约束(17)：保证所有放入容器 j 的物品总重不超过该箱的最大承重。

$$\sum_{i \in I} a_i p_{ij} \leq A_j u_j, \forall j \in J \quad (2.20)$$

约束(18)：同温区约束，若生鲜产品对温度敏感，则相同温区的物品只能放在相同容器内；如果两个物品 i 和 k 的温度需求不同，则不能放在同一个容器 j 中。

$$p_{ij} + p_{ik} \leq 1, \forall i, k \in I, T_i \neq T_k, \forall j \in J \quad (2.21)$$

约束(19)：支撑稳定性约束。

在三维装箱过程中，装载结构的稳定性是保证运输安全、减少货损、提高堆叠合理性的重要考量因素。特别是在生鲜物流等易碎物品的实际场景中，若上层货物仅部分悬空于下层之上，极易引发倾斜、滑落等问题。因此，本文在装载状态评估与解可行性判断中引入了稳定性约束条件，对每个被放置物品的支撑情况进行判断，确保装载方案满足基础物理稳定性。为此，本文设计了两层稳定性检验机制：

第一层检查：判断某一物品*i*所接触的下层物品是否具有足够的面积支撑其体积，即满足：

$$\sum_{z_j+h_j=z_i} s_{ij} \geq \alpha_i l_i w_i h_i, \forall i \in I \quad (2.22)$$

式中， s_{ij} 表示下层物品*j*与物品*i*接触的支撑面积， l_i, w_i 为物品*i*的底面长宽， $\alpha_i \in (0,1]$ 为最小支撑面积比例系数。

第二层检查：进一步约束每个下层物品所承受的最大支撑面积，以避免因受力过大导致下层失稳或压损。其数学表达如下：

$$\max_{z_j < z_i, \forall j \in J} s_{ij} \leq \beta_i l_i w_i h_i, \forall i \in I \quad (2.23)$$

式中， β_i 为最大可承受支撑面积比例系数，避免某一局部支撑面积过大。

约束(20)：二进制变量和非负性约束。

$$p_{ij}, u_j, x_{ik}^+, x_{ik}^-, y_{ik}^+, y_{ik}^-, z_{ik}^+, z_{ik}^-, l_i^x, l_i^y, w_i^x, w_i^y \in \{0,1\} \quad (2.24)$$

$$x_i, y_i, z_i \geq 0 \quad (2.25)$$

2.3 本章小结

本章首先明确了生鲜产品装箱问题的实际背景，并将其抽象为具备多重约束条件的三维装箱优化模型。针对冷链场景的特殊需求，本文在传统 3D-BPP 的基础上引入了承重、支撑、旋转限制等实际约束，提升了模型的适用性与复杂度。随后，系统定义了模型所需的参数、变量与坐标体系，并建立了装箱优化的双重目标函数。在此基础上，本文提出了更贴近实际的装载率定义方式，有效规避传统指标的偏差问题，为后续算法设计与实验分析奠定了基础。

第三章 算法设计

3.1 算法总体框架

本章节主要介绍算法的整体框架，并详细介绍算法的输入输出参数。

3.1.1 问题定义与输入输出

本章将详细阐述本研究所提出的基于树搜索与同层装载策略相结合的三维装箱优化算法的设计思路、结构模块及实现机制。该算法在充分利用启发式搜索与结构化块组合的基础上，引入适应度函数、多尺度评价机制及支撑性判断逻辑，以实现实际物流场景中复杂装箱问题的高效求解。

算法的输入主要包括：

（1）物品信息。输入数据中包含所有待装载物品的尺寸信息，每一个物品对象包含其原始长（`origin_l`）、原始宽（`origin_w`）与高（`origin_h`），此外还记录其体积信息、易损性、所属客户号和是否已被装载的状态。

（2）容器信息。容器具有固定尺寸，分别为最大装载长度（`max_load_length`）、最大装载宽度（`max_load_width`）以及最大装载高度（`max_load_height`），上述三个参数共同定义了空间的边界条件。

（3）算法控制参数。为引导装载过程的合理性，算法中设置了一系列关键性控制参数，包括最小支撑面积比例（`min_support_area_rate`）、最大投影面积比例（`max_projected_area_rate`）、装载率阈值（`load_rate_same_layer`）、随机扰动因子（`random_factor`）等。

算法的输出结果则为最终的装载方案，包含以下几项信息：

（1）每个物品的装载位置：通过 `Location (x, y, z)` 坐标描述。

（2）最终装箱方案：由若干个 `PackingSet` 组成的装载状态（`PackingState`），描述装载顺序与所使用容器个数，以及每个容器内的装载率；

（3）适应度（`fitness`）函数：衡量装载方案优劣的核心依据。

3.1.2 重要数据结构定义

在本研究提出的基于树搜索的三维装箱算法中，为实现装箱过程的建模与高效求解，需借助一系列数据结构对物品状态、空间划分、装载方案及搜索节点进行形式化表示。这些结构既是算法程序设计的基础单元，也是状态转移与装载评

估过程中的核心对象。为增强后续算法模块与实验分析的逻辑可读性，现对所用关键数据结构进行系统性定义如下。

（1）箱子（Bin）：用于装载物品的标准化容器，其尺寸固定，为三维装箱问题中的基本承载单元。每个装箱方案对应一个或多个 Bin 实例。

（2）物品（Item）：指待装载的单个货物体，具有固定的长度、宽度、高度等几何属性，同时可扩展承重、温度敏感性等属性以适应生鲜物流场景的建模需求。

（3）复合块（Block）：为提升空间利用率，多个物品可组合构成一个较大的复合块，称为 Block。Block 可分为旋转块（单个物品在空间中旋转）与组合块（多个物品结构性叠合）两类，是本算法中的装载单位。

（4）子空间（Space）：在装箱过程中，随着物品的逐步装入，初始空间被不断划分为多个剩余可用子空间。Space 对象用于记录当前可用于放置 Block 的空间位置及尺寸信息，是空间搜索与装载可行性判断的依据。

（5）单箱方案（PackingSet）：用于表示某一容器内的具体装载情况，包括已装载的 Block 列表及当前剩余可用空间集合。一个 PackingSet 对应一个 Bin。

（6）装箱方案（PackingSolution）：由一个或多个 PackingSet 构成，表示整个场景下多容器的联合装载结果，用于计算装载率、容器数等全局性能指标。

（7）装载状态（PackingState）：用于描述树搜索过程中任意搜索节点的状态信息，由当前装箱方案（PackingSolution）及待装载 Block 列表构成，是树结构中每一节点的状态载体。装载状态是树搜索算法中的扩展与评价基本单元。

（8）适应度（Fitness）：用于衡量装载状态的优劣程度，是搜索算法中的目标函数。本文设计的适应度函数综合考虑了装载率、使用容器数及装载顺序惩罚因子等要素。

图 3-1 展示了 PackingState 数据结构的示意图，其中左侧为已装载的 Block 所构成的三维堆叠结构，右侧为待装载 Block 的列表。该结构清晰刻画了装载过程中的状态构成，为后续装载状态扩展、评价函数计算及解空间搜索奠定了基础。

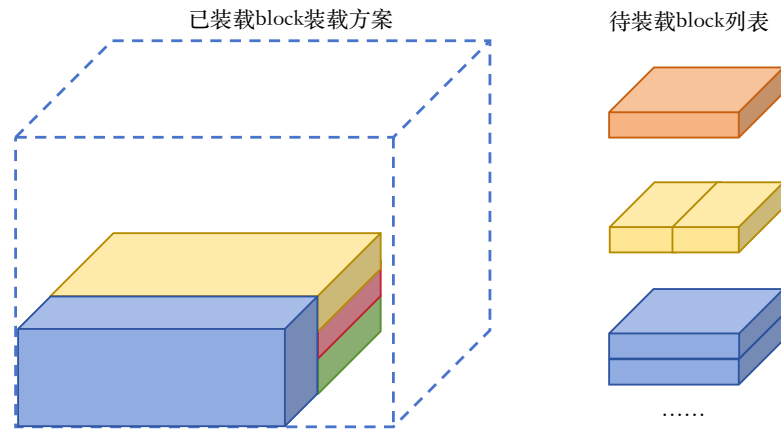


图 3-1 PackingState 示意图

3.1.3 算法整体框架

给定物品集合 `boxes` 和容器约束参数 `container_params`，算法整体分为四个主要阶段，分别为块构建与初始化、同层装载生成初始解、树搜索优化、方案评估与输出。各阶段具体执行过程如下所示：

Step 1: 货品定序初始化。货品定序算法首先将配送路径顺序逆序排列，实现“先送后装”，保证先配送客户的货品靠近容器外层。对于同一个客户的物品，按照易损性排序，非易损性物品优先装在容器底部，易损物品放后面装。

Step 2: 块构建。首先调用 `generate_blocks()` 函数，根据输入的箱体尺寸信息，构造出一系列结构块（Block）。其中包括原始箱体、旋转箱体构成的简单块，以及多个箱体在长度、宽度、高度方向拼接组合的复合块。所有生成的块按照高度进行划分，组织为高度字典结构，供后续同层装载策略调用。

Step 3: 同层装载策略生成初始解。调用 `get_same_layer_loaded_packing_state()` 函数，根据块的高度相似性构造层级结构，在每一层中尝试装载尺寸相近的块。装载过程中会考虑可用空间结构、支撑关系及接触稳定性等因素，确保每一层的放置既紧凑又稳定。最终形成初始装载状态 `initial_state`，作为树搜索的起始解。

Step 4: 树搜索优化解结构。进入 `beam_search()` 阶段，算法以 `initial_state` 为起点，逐步扩展状态空间。在每轮搜索中，选出当前得分前 `width` 个 `PackingState`，并调用 `extend_state()` 函数对其进行状态扩展。每次扩展会重新评估空间中剩余未装载块的放置可行性、适应度值，并更新状态队列，直至达到时间限制或宽度限制。

Step 5: 方案评估与结果输出。所有状态扩展完成后，将通过 `evaluate()` 函数对最终解状态进行打分，计算其 `fitness` 值。适应度函数会综合考虑装载率、容器使用数量、卸货分散度等指标，确保装载方案兼顾效率与实际可行性。随后调用 `make_box_loaded()` 函数补全所有箱体的空间位置信息，为最终输出或三维可视化做准备。

该算法步骤所对应的流程图如图 3-2 所示。

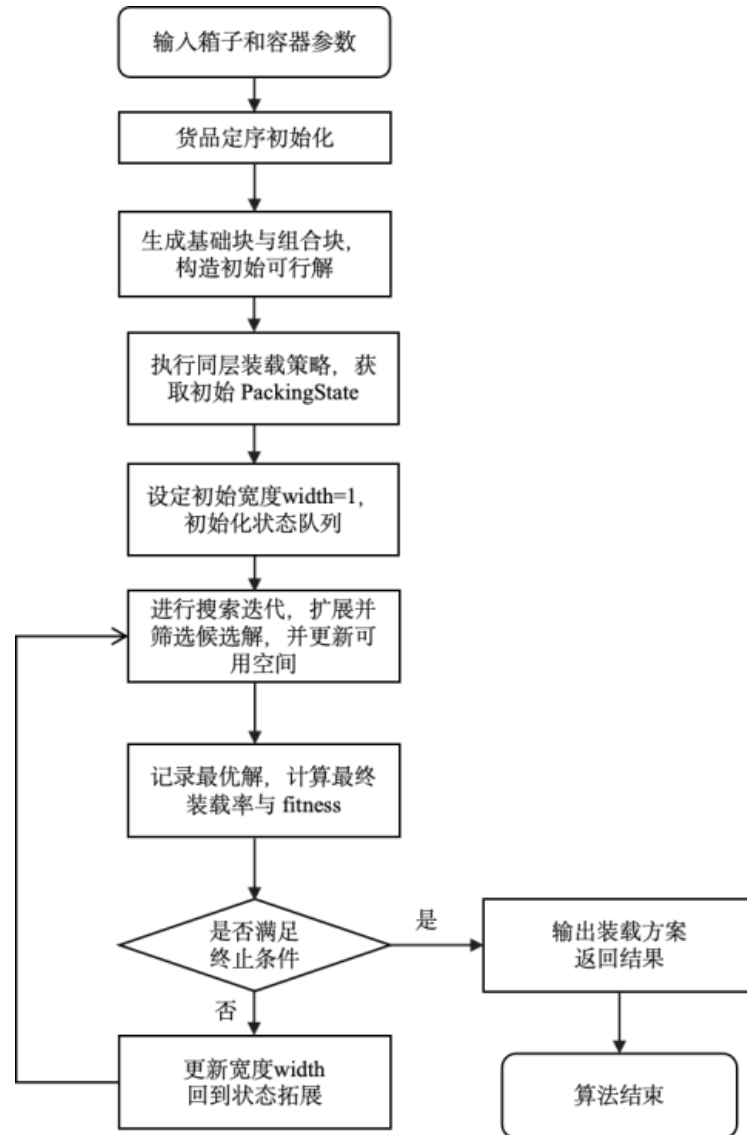


图 3-2 装箱算法流程图

3.2 算法模块功能设计

3.2.1 货品装载定序初始化

(1)定义路径装载顺序：输入路径顺序 $path=[1,2,3,4]$ ，然后定义货品装载顺序为路径顺序的逆序 $load_path=path[::-1]$ ，即颠倒输入的路径顺序，实现先装载的货品放置于容器里层，后装载的货品放置于容器外层，从而确保先配送的客户点所需物品能放置于容器外层，便于优先卸货。

(2)数组的初始化：初始化 $goods$ 与 $fragile$ 变量，形成空数组便于后续存储数据；初始化计数数组 $count$ 为长度等于路径长度的零数组，用于统计每个路径点的货品数量。

(3)分组与找寻装载货品：对于每个路径点 $load_path(i)$ ，首先筛选出所有目的地为该路径点的货品。随后，对于同一个客户的物品，按照易损值进一步排序，若某货品被标记为易损（ $fragile$ 属性为真），则将其索引或对象添加到易损品数组 $fragile$ 中，否则加入普通货品数组。

(4)合并所有分组：依次处理所有路径点和易损品分组后，最终得到综合考虑路径顺序、易损性的装载货品顺序 $goods$ ，可直接用于后续的装箱算法。

3.2.2 块生成机制设计

针对分组后的货品，分别进行块生成。考虑到多箱体组合结构在装载中的优势，算法首先将原始箱体组合为更大的装载单元“块”（Block），以提升后续搜索效率。块生成流程示意图见图 3-3，整体生成流程包含两个部分：

(1)简单块生成。每个输入箱体根据其可旋转性生成一个或两个简单块，包含原始方向和旋转 90 度方向，均保留其体积、相对位置等属性。

(2)复合块生成。使用宽度、长度、高度三个方向逐对组合已有块，满足组合体积占组合空间比例 $min_composite_flr$ 、不超过容器尺寸的前提下，将新块添加至候选集 B 中。通过分层迭代方式逐步扩展新块集合，并控制块总数不超过 max_block_num 。

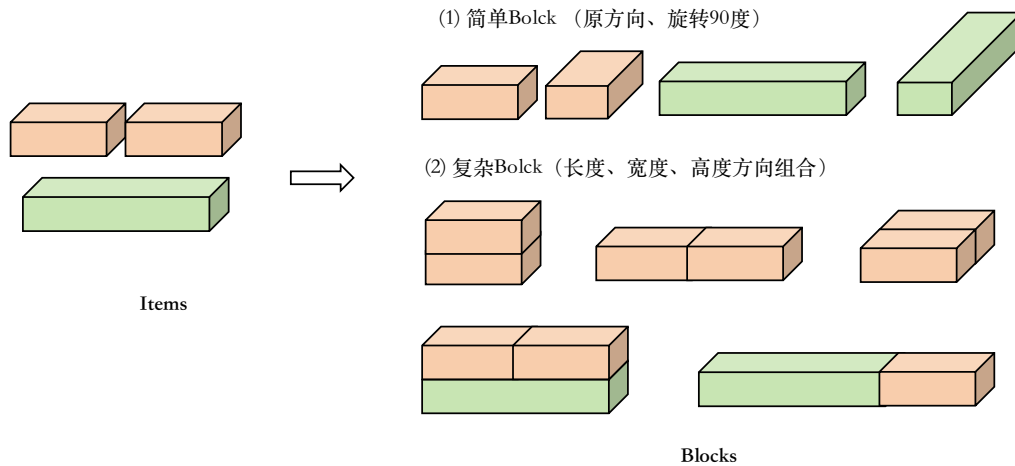


图 3-3 块生成流程示意图

所有生成块按照高度分组形成字典结构 `block_dict`，并记录高度列表 `heights` 以支持后续分层装载。

3.2.3 同层装载算法

第二个阶段，就是在第一个阶段块生成的基础上，进行以层为单位的三维装箱。该模块用于生成初始解，即依据块高度将箱体分层并逐层装载，其主要思想是：将高度接近的块（Block）组合在同一装载层内进行装载，以提升初始解的质量和后续搜索效率。该模块由函数 `get_same_layer_loaded_packing_state` 实现，其输入为预先生成的块集合字典 `block_dict`（按照高度进行分组）以及所有块的高度集合 `heights`。输出为一个初始的 `PackingState` 对象，包含当前所有已装载块、未装载块、以及对应的装载空间信息。

首先，在进行同层装载时，会根据当前容器的剩余高度逐层建立空间层（Layer），并判断当前高度层内是否存在装载效率较高的块组合。程序会通过 `get_unloaded_blocks_of_similar_height` 函数识别出高度差异在允许阈值 `height_diff_in_one_layer`（如 80mm）内的块作为候选集合，然后对每一个候选高度尝试构建一个装载空间 `Space`，该空间的底面为容器的当前顶面，长宽为容器尺寸，Z 轴起点为上层最高块的顶部坐标，厚度则为当前块集合的最大高度 `h`。

为确保所选高度层具备良好的装载质量，系统会枚举每一种可能的高度组合，并利用 `make_best_loaded_in_same_layer` 函数计算该层的装载率。具体而言，会将每一个未装载块尝试放入当前空间中，如成功放入，则记录当前的已装载块与装载体积，并继续尝试装载剩余块。最终选取当前层内装载率最高的一组块集合

作为最佳装载层。若某一组合的装载率高于阈值 $\text{load_rate_same_layer}$ （例如 85%），则该组合将被接受并更新当前 **PackingState**；否则，该高度层将被跳过，相关块保留在未装载状态。

同层装载结束的标志是在所有可接受的高度层中均无法找到符合要求的块组合，或者容器高度已接近上限。在完成装载后，程序会为该层顶部设置一个新的剩余空间 **Space**，其起始坐标为当前最高层 Z 轴坐标，形状大小为容器剩余部分的立方体，以为后续的树搜索阶段做准备。

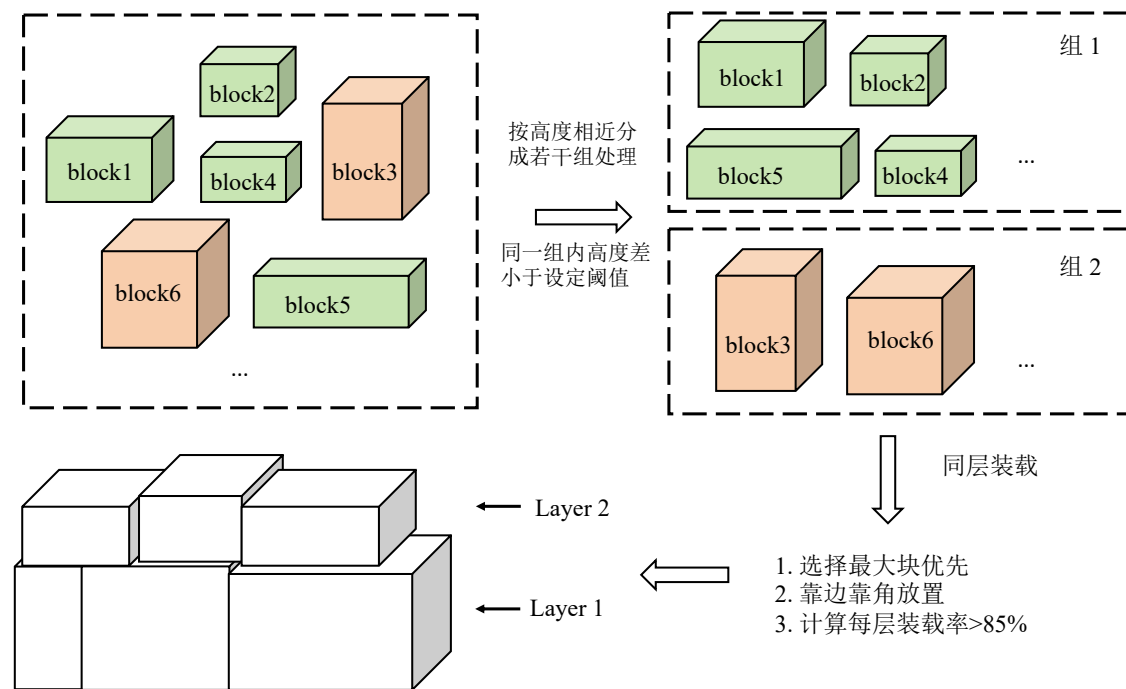


图 3-4 同层装载策略流程示意图

同层装载策略流程示意图见图 3-4。这种按高度划分空间层、在每层内集中摆放相似高度块的策略，本质上将一个复杂的三维装载问题拆解为若干个较小规模的子问题，即对每一个高度层单独求解最优块组合与放置方式。这样不仅有效降低了问题的求解难度，还提升了装载过程的局部最优性。此外，由于同层块往往在尺寸与属性上具有一定的相似性，该策略也更符合实际生产中的操作偏好，如将同种类或同批次物品集中装载于同一空间区域，有助于后续搬运、分拣与出库的作业效率提升。

3.2.4 树搜索优化策略

为进一步提升装箱效率，本文采用树搜索策略对装载状态空间进行剪枝优化与迭代探索。该策略以启发式评估函数为引导，通过限制每轮搜索的状态扩展宽

度，在保证解质量的同时显著降低了搜索复杂度与计算开销。其核心思想为：从初始状态出发，逐层迭代生成有限数量的高质量候选状态，构建一棵结构清晰、路径明确的状态搜索树。伪代码如下：

树搜索优化策略伪代码

```
While (状态队列 state_queue 非空) {
    对当前状态队列中的所有状态按适应度值排序;
    For 每个状态 state in state_queue {
        If (state 中所有块已装载完成) {
            记录当前装载率与适应度;
            若优于当前 best_state, 则更新;
            Continue;
        }
        调用 extend_state(state, width) 函数, 获取扩展状态集合
        extended_states;
        For 每个新状态 (键值对) (key, value) in extended_states {
            若 value 优于 best_state, 则更新;
            若 key 已在 next_states 中且 value 更优, 则替换;
            否则添加到 next_states;
        }
    }
    按适应度值降序选择前 width 个状态更新 state_queue;
}
Return best_state;
```

上述主算法负责整体装载状态的迭代优化，在每一轮状态搜索过程中，需调用子函数 `extend_state` 对当前状态进行扩展，生成多个新的候选装载状态。该子过程的核心目标是：根据当前容器剩余空间，在候选块集合中选取与之最匹配的若干块，并尝试将其合理装入，从而形成多个新的可行装载解。具体操作流程如下所示：

状态扩展子过程伪代码

```

While (state 中还有未装载块) {
    获取当前容器内剩余装载空间空间列表 spaces;
    For 每个空间 space {
        调用 get_ranked_blocks_by_vcs 获取前 width 个候选块;
        If (候选块非空) {
            For 每个候选块 {
                克隆当前状态为 new_state;
                在 space 中尝试放置该块;
                模拟剩余块填充, 计算适应度值 fitness;
                若更优则加入 extended_states;
            }
            Break;
        }
        当前空间无法装载, 尝试下一个;
    }
    若所有空间均无法装载, 创建新的容器并初始化空间;
}
Return extended_states;

```

该子过程作为主算法每一轮状态拓展的核心模块, 其执行效率与策略设计对最终解的质量与搜索耗时具有直接影响。

3.2.5 适应度函数设计策略

为对装载方案进行客观、全面的评估, 本文设计了多指标融合的适应度函数 fitness, Araya 等[11]针对存在物品选择的 SCLP, 提出新的适应度函数来确定物品排序, 该函数涉及物品体积与集装箱预估浪费容积等指标, 考虑之前已装箱货物影响来得到最优的装箱物品排序, 并将适应度函数融入 BSG 算法求解。并在算法执行过程中实时更新其数值。适应度函数的构成主要包括两部分: 加权平均装载率奖励项、容器使用数量惩罚项卸货。本算法采用如下综合评价策略:

(1) 当前装载率计算: 对当前 PackingState 的首个 PackingSet, 采用如下公式计算装载率。

$$load_rate_i = \frac{V_{loaded}}{L \cdot W \cdot H_{max}}$$

（2）加权装载率计算：考虑全部容器使用情况，每个容器的装载率 $load_rate$ 会按“先装优先”原则赋予不同的权重，即编号越小的容器装载率权重越大。也就是说，第 i 个容器的装载率乘以 $10^{(n-i-1)}$ ，其中 n 为容器总数。所有权重装载率之和构成适应度函数的基础得分，并乘以奖励参数 $load_reward$ ，代表高装载率对应高分。

$$R_w = \frac{\sum_{i=1}^n r_i \cdot w_i}{\sum_{i=1}^n w_i}$$

（3）容器数量惩罚项构建：为防止无节制地开启多个容器，函数会根据容器数量引入惩罚项，每多开启一个容器，适应度降低一个固定值 $fix_container_cost$ 。此外，为鼓励物品在同一容器中合理集中而非分散装载，算法还根据每个容器中非首个容器所含物品的数量引入卸载惩罚项，乘以单位惩罚因子 $unload_penalty$ ，最终加入适应度总值。综上， $fitness$ 函数表达式如下：

$$fitness = \left(\frac{\sum_{i=1}^n r_i \cdot 10^{n-i}}{\sum_{i=1}^n 10^{n-i}} \right) * load_reward - n * fix_container_cost$$

适应度 $fitness$ 函数设计既保证了装载率最大化目标，又平衡了实际装箱数量的现实需求。

3.3 本章小结

本章提出并设计了一种基于树搜索与同层装载策略相结合的启发式装箱算法。首先构建了算法的整体框架，明确了装箱问题的输入输出信息及各模块间的调用关系。随后构造原始块与复合块，设计了同层装载策略，根据块的高度相似性构建分层结构，并对状态空间进行剪枝扩展与优化，提升了装载效率与搜索质量。最后，设计了融合装载率、容器数量及分散惩罚的适应度函数，作为搜索过程中的评价指标和优化导向。

第四章 数值实验及结果分析

4.1 使用算例说明

本章基于所提出的同层装载策略树搜索算法，设计多组算例，以探究在不同箱型组合下，物品异质性对装载率的影响。所有算例均在笔记本电脑（MacBook Air，Apple M3 芯片，8 核 CPU + 10 核 GPU，16GB RAM）上完成求解与测试。

为探究物品异质性对装载率的影响，本文设计了三类物品尺寸组合，分别代表低异质性（类型 1）、中异质性（类型 2）和高异质性（类型 3），这三类算例在生成物品尺寸时，依次扩大其长度、宽度和高度的取值范围，以此模拟现实中从标准化箱型到完全非标准化箱型的不同装载复杂程度。其中，异质性程度的提升，意味着箱型之间的尺寸差异更大、组合方式更复杂。由于目前国内外尚无统一的标准测试算例，本文采用随机生成的方式构建数据集。各类型物品尺寸及对应容器参数如表 4.1 所示。

表 4.1 随机生成的测试算例说明

类型	长	宽	高	容器尺寸（L，W，H）
1	[0.4W,0.5W]	[0.25W,0.3W]	[0.15W,0.3W]	（1100，1100，1500）
2	[0.3W,0.5W]	[0.2W,0.5W]	[0.1W,0.3W]	（1100，1100，1500）
3	[200,0.5W]	[70,0.5W]	[100,0.5W]	（1100，1100，1500）

在容器参数设置方面，统一采用符合日本标准托盘规格的箱体尺寸即 L=1100mm，W=1100mm。

4.2 实验结果及其分析

本文针对三类异质性水平分别进行了多个装箱算例的测试，并对各类结果求平均，统计结果如表 4.2 所示。可以看出，随着物品异质性逐渐提高，装载率呈下降趋势，其中“单一箱型”装载率最高达 95%，“高异质性”场景下则降至 76%。总体来看，所有类型的平均装载率为 86%。

表 4.2 装箱算法结果

类型	装载率 (%)	容器尺寸 (L, W, H)
单一箱型	95	(1100, 1100, 1500)
中异质性	87	(1100, 1100, 1500)
高异质性	76	(1100, 1100, 1500)
所有类型	86	-

接下来的章节将分别展示不同类型装箱算例的典型可视化结果，并进一步分析影响装载率变化的因素。

4.2.1 单一货品装箱展示

为了进一步分析算法在低异质性条件下的表现，本文以单一货品类型为例，选取其中一个典型算例进行展示。设定容器尺寸为 $L=1100\text{mm}$ ， $W=1100\text{mm}$ ， $H=1500\text{mm}$ ，随机生成物品规格为 $l=530\text{mm}$ ， $w=275\text{mm}$ ， $h=290\text{mm}$ 。图 4-1 展示了该算例下的装箱结果。

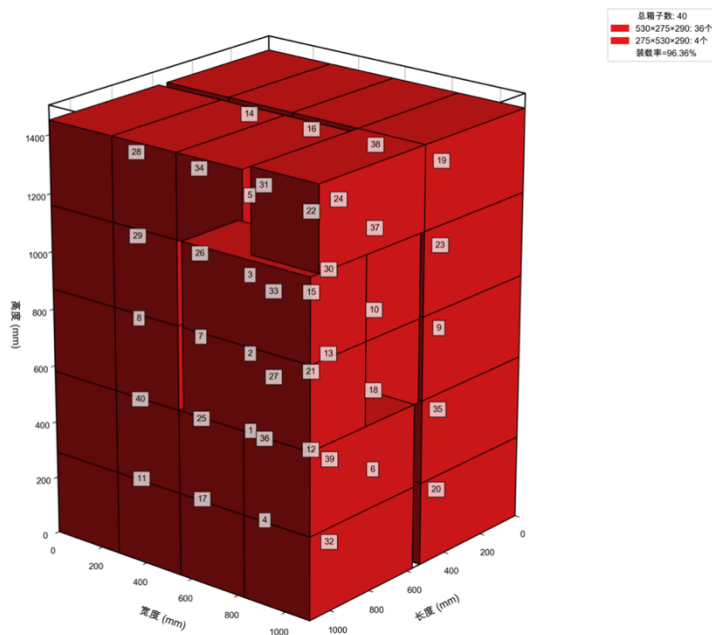


图 4-1 单一类型装箱结果示意图

从图中可见，物品在三维空间中排布紧凑，容器几乎被完全填满，最终装载率达到 96.36%。该结果表明在物品尺寸与容器尺寸匹配度较高的情况下，算法能够实现较优装载效果。但在长宽比例不协调、即物品尺寸与容器尺寸偏差较大

时，空间利用率会明显下降。经过多组单一类型货品的测试，整体平均装载率约为 95%，显示出该类问题在实际场景下仍具有较好的适应性。

4.2.2 中等异质性货品装箱展示

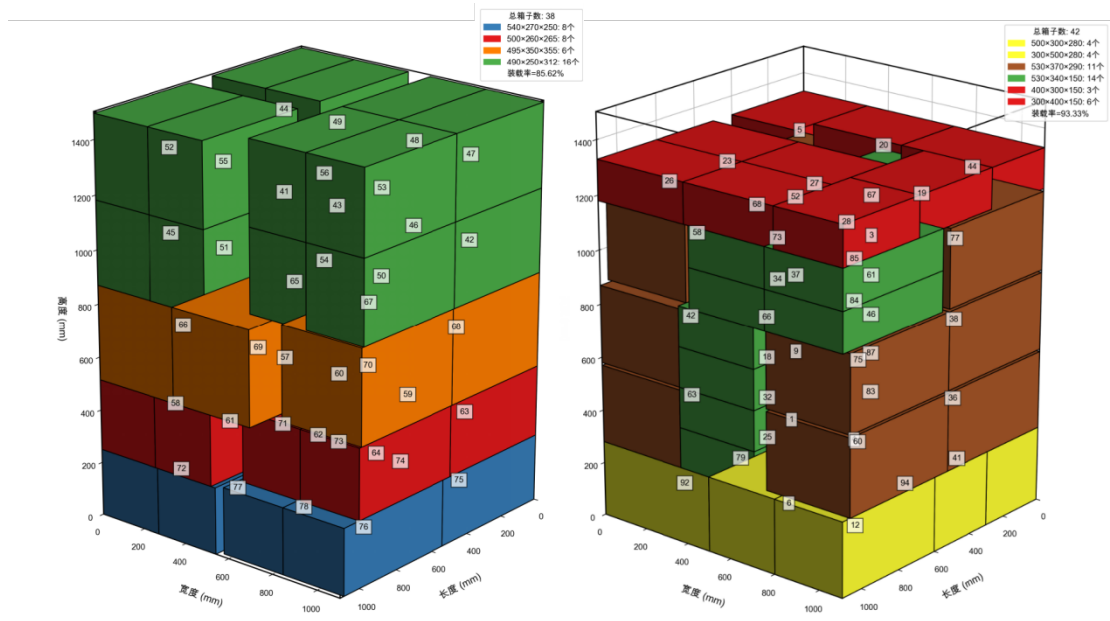
本节选取中等异质性的典型装箱案例进行展示。设定容器尺寸为 $L=1100\text{mm}$ ， $W=1100\text{mm}$ ， $H=1500\text{mm}$ ，随机生成多种规格的物品箱型。算例（a）与（b）的物品规格信息如表 4.3 与表 4.4 所示，装箱过程中满足“易损品必须放置于非易损品之上”的摆放约束，装载结果如图 4-2 中图（a）（b）所示。

表 4.3 物品规格信息（a）

长	宽	高	数量	是否易损
540	270	250	8	否
500	260	265	8	否
495	350	355	6	否
490	250	312	16	是

表 4.4 物品规格信息（b）

长	宽	高	数量	是否易损
500	300	280	8	否
530	370	290	11	否
530	340	150	14	否
400	300	150	9	是



(a) 装载率 85.62%

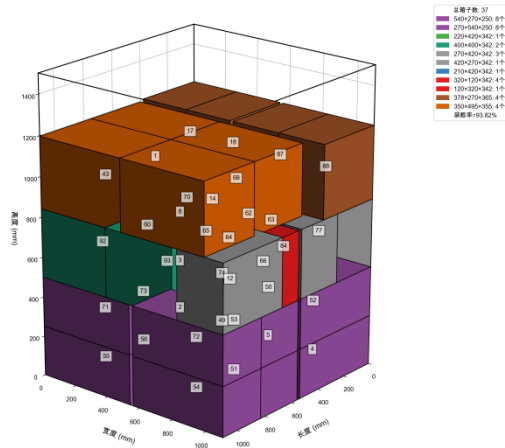
(b) 装载率 93.33%

图 4-2 4 种箱型时装箱结果可视化

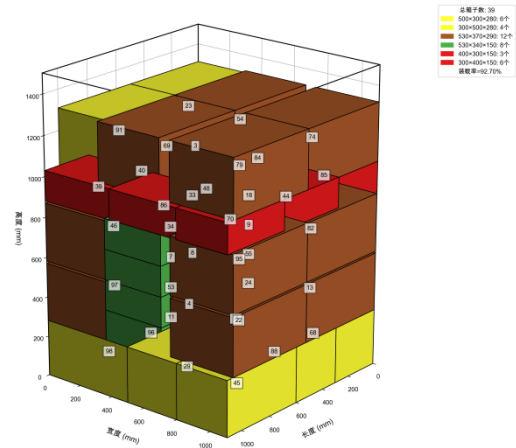
图 (a) 与图 (b) 分别展示了两组中异质性场景下的装箱结果，装载率分别为 85.62% 与 93.33%。从图中可以看出，尽管箱型数量较多、规格不一，但在满足易损品堆叠约束的前提下，仍可获得较高的装载率。图 (a) 中的箱型尺寸相对均衡，绿色箱型在顶部形成规则堆叠，有效利用竖向空间，而底层以红色与蓝色箱型铺底，形成较为规整的装载结构。但部分中层箱体之间存在空隙，导致空间利用存在浪费，装载率略低于图 (b)。

图 (b) 中的装载效果显著更优，装载结构紧凑，整体布局稳定，尤其是红色、棕色与绿色箱型之间形成良好的互补关系，增强了层间支撑性，从而提升了整体装载率至 93.33%。这说明合理的箱型搭配和有效的空间填充策略对提升装载率具有显著影响。

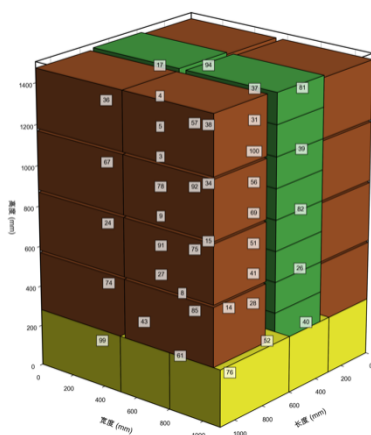
此外，为进一步分析算法在中等异质性场景下的表现，本文还补充展示了图 4-9 中其余六组装箱结果，其装载率依次为 93.82%、92.70%、93.72%、80.29%、80.74%、70.33%。



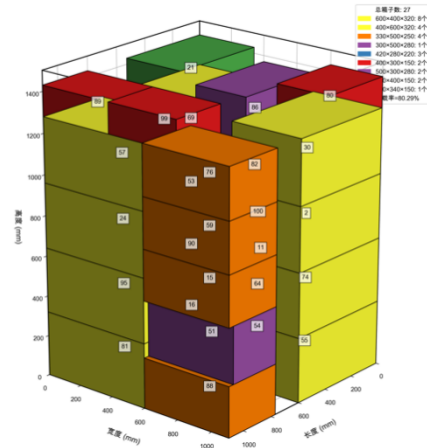
(c) 装载率 93.82%



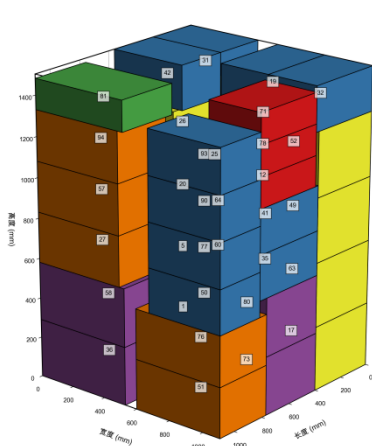
(d) 装载率 92.70%



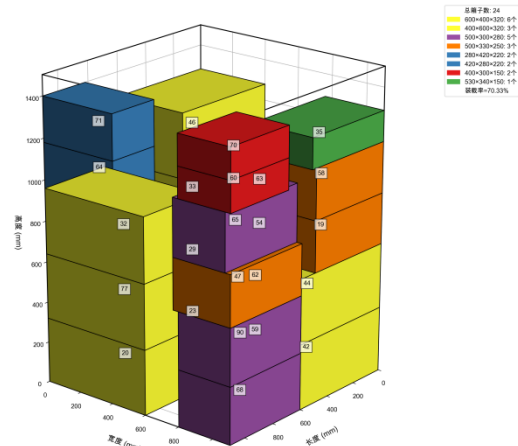
(e) 装载率 93.72%



(f) 装载率 80.29%



(g) 装载率 80.74%



(h) 装载率 70.33%

图 4-3 中等异质性多箱型装箱结果可视化

在图 (c) (d) (e) 中, 装载率均超过 92%, 表现出优异的空间利用能力。其中, 图 (c) 包含 7 种箱型, 通过合理的堆叠顺序实现了空间的高效填充。紫

色箱体形成了稳定的底层，橙色与棕色箱体构成中层，顶部由绿色与灰色箱体紧密封顶，层层相叠、过渡自然，有效减少了竖向与横向空隙，使装载率达到 93.82%。

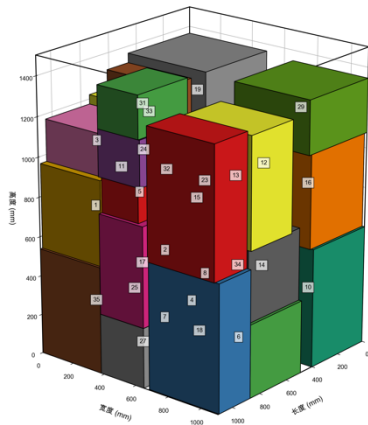
图(d)中的箱体以黄色和棕色为主，红色与绿色箱体作为过渡填补层间空隙，实现结构均衡。尽管箱型数较多，但主导箱型尺寸相对一致，形成规则整层结构，装载率高达 92.70%。图(e)则以棕色箱体为主导，在底层与中层形成连续体积支撑，上部由绿色和黄色箱体填补剩余空间，展现出较强的层间耦合性，装载率达到 93.72%。

相对而言，图(f)(g)(h)中装载率出现明显下降，空间利用效果较弱。图(f)中虽存在大型箱体(黄色)，但由于其他颜色箱体高度不一，导致层间不连续，顶部堆叠呈现碎片化结构，装载率下降至 80.29%。图(g)展示了最多达 9 种箱型的装载结构，箱体颜色多样，但尺寸协调性不足，尤其顶部空间堆叠较为混乱，出现较多悬空与交错，造成垂直方向空间浪费，装载率为 80.74%。图(h)中装载结构最为松散，箱型尺寸差异过大，未形成稳定的连续装载层，多个位置存在空腔，未充分利用角落与顶部空间，最终装载率仅为 70.33%。

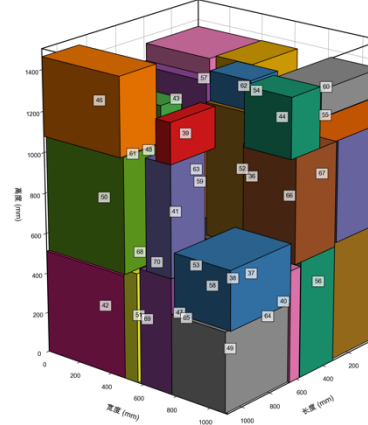
综上可知，高异质性直接影响装载结构的完整性与空间连续性。当箱型之间缺乏统一比例关系，尺寸变化幅度过大时，算法难以在有限时间内构建理想的组合块结构，容易导致空间碎片化、底层空置和顶部堆叠不稳等问题，显著拉低整体装载率。

4.2.3 高异质性货品装箱展示

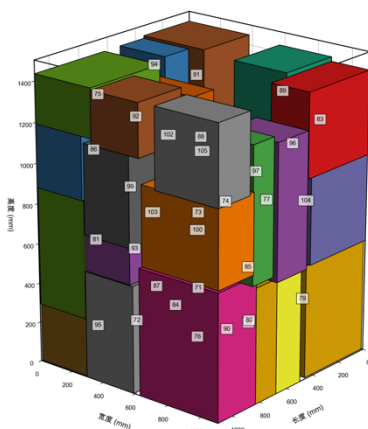
本节选取高异质性货品的典型装箱结果进行展示与分析。设定容器尺寸为 $L=1100\text{mm}$, $W=1100\text{mm}$, $H=1500\text{mm}$ ，物品箱型种类较多，每组算例包含约 30~35 种不同规格，且箱型尺寸分布广泛，异质性显著。采用本文提出的基于同层装载策略的树搜索算法进行求解，选取 8 组具有代表性的装箱结果图如图 4-4 所示。



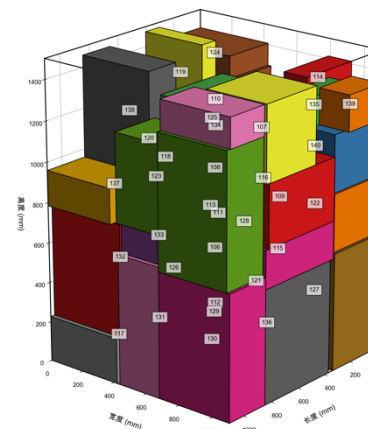
(a) 装载率 76.69%



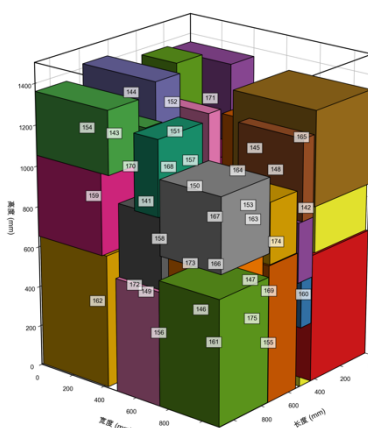
(b) 装载率 71.76%



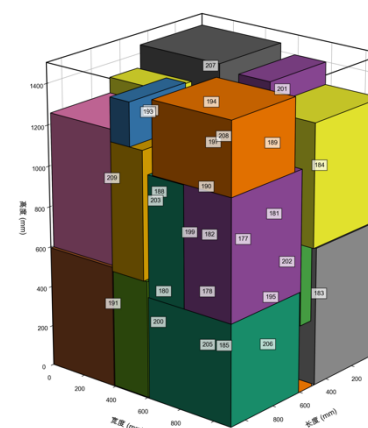
(c) 装载率 77.73%



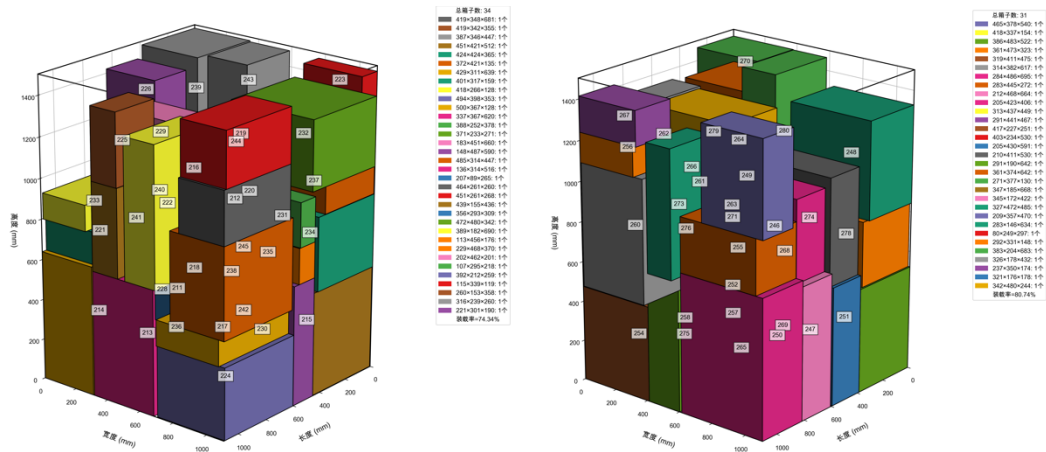
(d) 装载率 79.48%



(e) 装载率 72.67%



(f) 装载率 79.85%



(g) 装载率 74.34%

(h) 装载率 80.74%

图 4-4 高异质性多箱型装箱结果可视化

从图中可以观察到，算例 1 至 8 的装载率分别为对应装载率依次为 76.69%、71.76%、77.73%、79.48%、72.67%、79.85%、74.34%、80.74%。整体来看，装载率波动较大，平均装载率为 76.66%，相比中等异质性场景下降约 10%，验证了箱型高度异构性对空间利用效率的明显抑制作用。

在图（a）中，箱体颜色较多，尺寸高度分散，部分高度大的箱体未能充分贴合顶部空间，导致垂直空间存在碎片空隙，装载率为 76.69%。图（b）中堆叠层级不连续，箱体体积差异较大，支撑结构薄弱，空间浪费现象较为明显，装载率仅 71.76%。图（c）中箱体整体高度协调性略有提升，底层构建较为稳定，边角空间填充效果一般，装载率为 77.73%。图（d）为表现较好的一组装箱结果，尽管箱型种类多达 35 种，但通过主导箱型的统一高度支撑，形成了良好的层级结构，空间利用效率较高，装载率达 79.48%。图（e）中虽然也具备一定的底层规整性，但中层与顶部的箱体尺寸匹配度不高，堆叠密实度下降，装载率回落至 72.67%。图（f）中装箱结构层级清晰，箱型组合呈现一定规律性，边角空隙较少，装载率为 79.85%。图（g）呈现出的装箱结构较为分散，虽包含多个长条状箱型进行边角填补，但未能有效支撑上层结构，整体装载率为 74.34%。图（h）为本组中装载效果最优的算例，部分箱型具备良好的层级叠加性，顶部空间利用充分，装载率达到 80.74%。

综上所述，高异质性场景下箱型尺寸差异大、边角空间复杂，显著增加了启发式算法的组合与匹配难度。

4.2.4 结果分析

本节通过对三类异质性水平（单一、中等、高）下的装箱实验结果进行归纳，发现物品异质性与装载率之间存在显著的负相关关系：当箱型种类从单一增加至高度异构，平均装载率由 95% 降至 76%，下降幅度近 20%。这一趋势在算法求解过程中表现出多重机制性原因，本文从以下几个角度进行深入分析：

首先，从算法搜索效率角度看，异质性越高，箱型组合方式呈指数级增长。对于同层装载策略而言，其核心在于基于块生成机制寻找当前层内的最优匹配结构。本文算法依赖块生成与同层装载策略，在低异质性场景中能够快速构建出规则块，实现高效堆叠；但在高异质性条件下，因箱型种类众多，组合块难以快速生成或无法复用，使得启发式算法需在多个不规则组合间不断尝试，计算复杂度与时间成本显著上升。尤其当尺寸差异过大、小箱体比例过高时，块结构难以稳定，易造成空间浪费，装载率大幅下降。图 4-4（a）（f）均出现了底部空腔、边角残余空间等现象，反映出块构建失败带来的结构松散问题。

其次，装载率与空间连续性与结构耦合性密切相关。从图 4-1 至图 4-9 的可视化结果来看，高装载率通常伴随整齐、规则、可层层支撑的结构特征。而当箱型差异过大时，容易导致层间断裂、顶部堆叠紊乱，形成大量竖向碎片空间。这一特征在图 4-3（f）（g）（h）和图 4-4（a）（b）（e）中表现明显，多个不同尺寸箱体堆叠高度不一，无法构建封闭支撑层，导致空间利用效率显著降低。

第三，装载结果与箱型结构特征高度相关。经过多组实验归纳总结，影响装载率的关键因素主要包括：

- 1) 箱型尺寸是否呈倍数或接近关系，决定是否能形成完整层结构；
- 2) 主导箱型是否具备稳定支撑能力，决定底层堆叠的稳固性；
- 3) 是否存在合适的小箱体用于边角填补，决定局部空间的利用效率；
- 4) 是否能在垂直方向形成连续堆叠层，影响整体结构的紧凑性。

4.2.5 管理启示

基于上述分析，本文提出以下四点管理启示：

- 1) 控制箱型种类数量。应合理控制运输任务中所使用的箱型种类，避免因规格过多导致的堆叠不稳定与空间浪费，从而降低算法处理难度，提升装载效果。

- 2) 推动箱型模数化设计。建议在长、宽、高方向上采用统一模数设计，即各箱体在尺寸上形成倍数关系或整数比例，有助于构建完整层结构，提升装载规则性和紧凑度。
- 3) 建立主导箱型配置机制。在物品装载结构中应设置一定比例的主导箱型，这类箱体具备较大的体积与良好的结构稳定性，可作为构建底层与中层的骨架，提高整体承载力。
- 4) 合理配置小型箱体进行边角填补。在设计物品组合方案时，应有意识地配置部分小尺寸箱体，用于填补大箱体堆叠后所留下的边角空隙，从而最大化容器空间利用率。

综上所述，装载率的提升不仅依赖于算法层面的优化，还需在物品包装与运输管理的源头环节进行系统化改进。通过推进箱型标准化与模数化设计，构建结构清晰、组合合理的装载基础条件，可有效增强启发式算法的求解能力，为冷链物流企业实现运输成本降低与装载效率提升提供重要支撑。

4.3 支撑面系数 α 敏感性分析

支撑面系数 α 表示在堆叠装载过程中，上层物品与下层物品之间的接触面积比例，是衡量装载稳定性的重要指标。对于易损性较高的生鲜产品，如水果类和软包装蔬菜，其在运输过程中的抗压能力较差，因此对支撑面的要求较高，以避免因受力不均导致的损坏。

分析装载率随支撑面系数 α 变化的情况。这里我们设置 $\alpha \in [0.5, 1.0]$ 进行数值实验，以 0.05 为步长，观测不同支撑面比例下装载效果与成本变化。

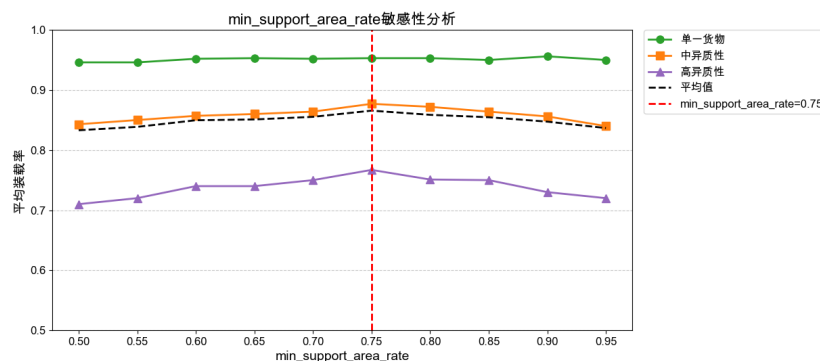


图 4-5 支撑面系数敏感性分析图

从图 4-5 中可以看出，随 α 从 0.5 增大到 0.95，三类场景均呈现“先升后降”的变化特征，但敏感程度存在明显差异。在单一箱型情境下，箱型尺寸一致，装载结构规则，装载率变化趋势不明显。而在中高异质性的情况下，平均装载率呈现出上升后下降的趋势。当支撑面系数增大后，装载策略需要更多空间来满足稳定性条件，尤其在支撑面要求接近 1.0 时，由于缺少装载灵活性，使得平均装载率变小。因此，当支撑面系数设置为 0.75 时，能够在保证货物堆叠稳定性的同时，最大程度减少空间浪费。

4.4 与 Gurobi 比较的方法有效性验证

为验证本文提出的基于同层装载策略的树搜索启发式算法的有效性，本文选取求解器 Gurobi（版本 12.0.1）作为对比基准，并统一设置最大求解时间为 600 秒，超时后输出当前最优解。启发式算法由 Python 3.11 实现，在 MacBook Air M3（Apple M3 芯片，8 核 CPU + 10 核 GPU，16GB RAM）笔记本电脑上运行。两种算法均在单线程模式下运行，以保证实验结果的可比性。其中，Gurobi 求解 MIP 模型，启发式算法通过遍历束内路径或满足提前终止条件完成求解。

为反映实际作业中水果多样性与包装多规格共存的情况，本文汇总并抽取 3 类典型水果包装箱型，构建测试数据集，覆盖了大中小不同体积层级，具备较强代表性，其三维尺寸、包装方式、是否易损如下表 4.5 所示。

表 4.5 常见典型水果箱尺寸参数表

水果	长/mm	宽/mm	高/mm	重量/kg	包装方式	是否易损
苹果	500	300	280	11	外层包装	否
橙子	500	330	250	10	外层包装	否
樱桃	400	300	150	6	内外双层包装	是

4.3.1 小规模算例对比分析

为验证本文所设计算法的可行性及有效性，按照常见典型水果箱尺寸参数表提到的箱型数据，随机生成了 10 组模拟算例，每组包含 60 个物品，假设容器尺寸固定为 $2000 \times 1500 \times 1500\text{mm}$ 。通过运行两种算法，以各算例的装载率与求解时间为关键性能指标。表 4.6 展示了树搜索和 Gurobi 算法的求解结果。其中，装载率单位为%，用时单位为 s。

表 4.6 两种算法比较

算例	Gurobi		树搜索		Gap
	装载率	用时	装载率	用时	
test1	92.46	300.58	88.75	104.84	4.39
test2	90.63	301.59	90.63	51.77	0
test3	94.54	300.19	92.72	43.93	1.93
test4	87.25	301.25	87.25	14.65	0
test5	97.53	301.33	92.65	21.3	5
test6	83.01	301.3	83.01	32.9	0
test7	93.34	301.33	89.52	90	4.09
test8	92.75	301.67	90.24	48.32	2.71
test9	92.9	301.28	89.53	82.92	3.63
test10	84.02	300.23	84.02	35.59	0
平均值	90.843	301.075	88.83	52.622	2.17

注：解差距 $\text{Gap} = (\text{Gurobi 装载率} - \text{树搜索装载率}) / \text{Gurobi 装载率} * 100\%$

（1）在解的质量方面：从表中可以看出，Gurobi 算法在多数算例中取得了更高的装载率，其平均装载率达到 90.83%，在 10 组测试数据中，有 2 组（test3 和 test5）装载率超过 94%。相较之下，树搜索算法的平均装载率为 88.83%，表现略劣，但在部分算例中依然展现出较强的解质能力，如 test2、test4、test6、test10 中装载率与 Gurobi 求解结果保持一致，且其余算例对应偏差均小于 5%，但从解的质量上来看，二者结果偏差不大。

（2）在求解效率方面：树搜索显著优于 Gurobi。Gurobi 所有算例均在 301 秒附近完成计算，计算开销较大。而树搜索的平均求解时间仅为 52.62 秒，约为 Gurobi 的 17%。树搜索算法在求解时主要基于规则和策略实现，求解效率较高，具备更快的响应速度。

整体来看，用 Gurobi 求解数学模型和用树搜索都能实现生鲜产品装载策略问题的有效求解，树搜索求解的质量略劣于 Gurobi 的结果。但是，对比数学规划方法，本文提出的基于树搜索与同层装载策略的三维装箱启发式算法的算法在解质量方面表现稳定，并且只需要一半的时间便能快速得到问题的高质量解。

（3）装载方案可视化展示。以算例 test1 为例，图 4-6 展示了基于树搜索算法得到的装载可视化结果。图中可见，底层与中层主要为体积较大的苹果与橙子，顶部堆叠的是较轻且易损的樱桃（蓝色物品），符合“易损品上置”的约束。整体结构紧凑、布局平稳，装载率达 88.75%，体现出较强的稳定性与空间利用能力。

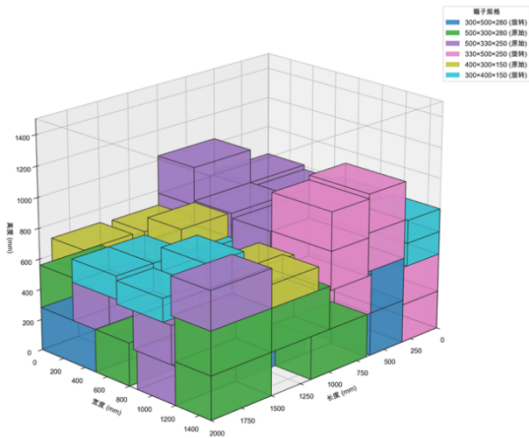


图 4-6 算例 1 装载效果图

4.3.2 大规模算例对比分析

为验证本文设计的树搜索算法在大规模算例中的表现，本文设计了 20 组模拟算例，前 10 组（test11-test20）包含 100 个物品；后 10 组（test21-test30）包含 200 个物品，分别用数学规划和启发式方法求解。此时，由于变量规模过大，Gurobi 已经无法顺利求解此模型，而本研究设计的树搜索算法表现稳定、性能优秀。此本节仅列出树搜索算法的结果，分析其在大规模下的求解能力。

表 4.7 树搜索在大规模算例中求解表现

算例	装载率	时间	算例	装载率	时间
test11	89.22	243	test21	88.32	412.5
test12	88.23	303.2	test22	93.23	531.02
test13	89.34	368	test23	89.57	453.94
test14	90.53	296.5	test24	88.86	450.47
test15	90.94	294.65	test25	90.47	463.63
test16	89.21	340.28	test26	83.53	360.17
test17	90.43	268.33	test27	87.13	396.06
test18	85.53	226.54	test28	89.42	520.61
test19	90.58	332.55	test29	90.24	467.88
test20	90.64	330.48	test30	87.56	440.43
平均值	89.465	300.35	平均值	88.833	449.67

在大规模算例的求解中，树搜索算法展现出良好的稳定性与高效性。

从装载率结果来看，表 4.7 显示，在 100 个物品的测试组中，算法平均装载率达 89.465%，而在 200 个物品的测试组中，装载率仅小幅下降至 88.833%，整体保持在较高水平，接近理论上的最优填充效果。这一表现也与前文中中异性场景下 87% 的平均装载率较为接近，说明即便装载规模加倍，算法依然能有效维持解的紧凑性，未出现明显性能衰减，体现出良好的扩展性和稳健性。

在求解时间方面，树搜索算法同样具备出色表现。在 100 箱规模下，其平均运行时间为 300.35 秒；当箱数扩大至 200 个时，求解时间增至 449.67 秒，呈温和线性增长趋势。尽管求解时间有所增加，但与传统精确方法 Gurobi 在同类问题中频繁遭遇求解超时或内存溢出的情况相比，树搜索仍表现出更强的实用性与求解能力。例如 test18 算例中，仅用 226.54 秒便完成 200 箱的装载计算，效率甚至超过 Gurobi 在小规模任务中的运行表现。

4.3.2 Gurobi 与树搜索算法对比分析小结

Gurobi 适用于精度要求较高的小规模（小于 60 个物品）装箱任务，在物品数量大于 60 个左右时便无法求解，其运行时间固定且受限于预设参数，灵活性较低；而本文提出的基于树搜索与同层装载策略的三维装箱启发式算法的算法在解质量方面表现稳定，不管是小规模还是大规模算例时，求解性能和精度均表现较好，装载率均在 88% 左右，与 Gurobi 的 91% 相差不大，解差距 Gap 为 2.17%，且整体求解速度快，求解时间约为 Gurobi 的 17%，表现出较强的实用性与高效性。

4.5 本章小结

使用树搜索的三维装箱算法结合同层装载策略与空间块生成机制，开展了系统的数值实验与结果分析，首先通过构建三类典型算例，分别为单一箱型、中异性、高异性，实证验证了箱型异性对装载率的显著影响。实验结果显示，随着物品尺寸分布的多样性增强，整体装载率呈下降趋势，由单一场景下的 95% 下降至高异性场景下的 76%，随后分析了影响装载率的原因，明确指出完整层结构、主导箱型支撑和边角填补等因素对装载效率具有直接影响。接着，对本文模型进行敏感性分析，观察装载率随着相关系数的变化，得到最优的支撑面系数取值，为企业管理者对相关参数的决策提供依据。最后，本文将 Gurobi 求解结

果与树搜索求解结果进行对比，得出在小规模算例，Gurobi 表现相对优秀，但求解较慢；本文提出的基于同层装载策略的树搜索在小规模（小于 60 个物品）和大规模算例中表现均较出色。

第五章总结与展望

5.1 论文总结

本文聚焦于生鲜装载场景，本研究构建了三维装箱数学模型，得到了装载优化方案。本文主要研究结论如下：

(1)建立了多目标的三维装箱模型。该模型在经典装箱模型的基础上，综合考虑了生鲜产品的易损保护、配送与卸货顺序、承重能力、物品支撑稳定性等约束条件，通过合理简化假设，最终建立了最小化容器总数、最大化装载率的多目标装箱模型。

(2)设计了一种基于树搜索的启发式算法，结合块生成与同层装载策略，提升初始解质量，并通过状态扩展与适应度函数动态筛选高质量装载结构。算法在实验中展现出良好的效率与适应性，适用于复杂生鲜装载场景。

(3)构建三类典型算例（单一箱型、中异质性、高异质性），结果表明箱型异质性对装载率影响显著，装载率由 95%逐步下降至 76%。进一步分析指出，完整层结构、主导箱型支撑能力与边角填补策略是提升装载率的核心要素，并基于此提出四点管理建议：控制箱型种类数量、推进模数化设计、配置主导箱型以及合理配置边角小箱体，从源头提升算法适应性与装载效率。随后，通过对支撑面系数 α 进行敏感性分析，确定了在 0.75 附近取值可在稳定性与装载效率之间取得较好平衡。

(4)进行算法有效性分析，与 Gurobi 求解器求解结果进行对比。结果显示：树搜索算法在小规模任务下解的精度与 Gurobi 相近（平均装载率差距约 2.17%），但求解时间仅为其 17%左右；在大规模装箱任务中，Gurobi 难以有效求解，而树搜索依然保持稳定性能，装载率维持在 88%左右，展现出良好的扩展性与实用性。

5.2 研究展望

本文在探索基于树搜索的三维装箱优化问题的基础上，仍存在进一步研究与拓展的空间，未来的研究可从以下两个层面展开：

（1）问题建模层面：为更贴合实际应用场景，可进一步引入或组合更多现实约束条件。例如，装载优先级约束可用于多点配送场景下的装卸顺序控制；重心平

衡约束有助于提高运输过程中的安全性与稳定性；物品绑定不可拆分约束适用于整箱打包、成套出货等业务模式。此外，面对物流企业日益增长的即时响应需求，在线装箱问题的研究将成为具有挑战性和应用前景的重要方向。

（2）求解方法层面：除本研究中采用的启发式搜索方法外，未来可尝试构建更具表达能力的三维装箱问题数学模型，并探索高效的精确算法或多阶段近似算法。在此基础上，融合强化学习等数据驱动方法亦是提升算法泛化能力和实时决策性能的有力手段，有望进一步推动算法在复杂环境中的适应性和实用性提升。

参考文献

- [1] 王凤丽, 杨信廷. 农产品物流配送过程中三维装载与车辆路径混合问题研究[D].华东交通大学, 2011.
- [2] 裴霞, 吴晓. 基于冷链物流的生鲜产品运输与配送问题研究[D].西南交通大学, 2021.
- [3] 赖华平, 杨正书. 基于 HACCP 和 QFD 的生鲜产品配送质量控制研究及应用[D].重庆大学, 2013.
- [4] 林云鹏, 江志斌. 电商物流背景下基于空间矩阵的三维装箱算法及箱型设计研究[D].上海交通大学, 2021.
- [5] Ali Sara, Ramos António Galvão, Carravilla Maria Antónia, Oliveira José Fernando. On-line three-dimensional packing problems: A review of off-line and on-line solution approaches[J]. Computers & Industrial Engineering, 2022, 168: 108122.
- [6] 智阳光, 王书振. 三维装箱与选箱算法研究[D].西安电子科技大学, 2021.
- [7] Hallawi Huda, Mehnen Jörn, He Hongmei. Multi-Capacity Combinatorial Ordering GA in Application to Cloud resources allocation and efficient virtual machines consolidation[J]. Future Generation Computer Systems, 2017, 69: 1-10.
- [8] Baker B S, Coffman, Jr. E G. A Tight Asymptotic Bound for Next-Fit-Decreasing Bin-Packing[J]. SIAM Journal on Algebraic Discrete Methods, 1981, 2(2): 147-152.
- [9] Ren Runtian, Tang Xueyan, Li Yusen, Cai Wentong. Competitiveness of Dynamic Bin Packing for Online Cloud Server Allocation[J]. IEEE/ACM Transactions on Networking, 2017, 25(3): 1324-1331.
- [10] Coffman E G, Garey M R, Johnson D S. Approximation Algorithms for Bin-Packing — An Updated Survey[M]//Ausello G, Lucertini M, Serafini P. Algorithm Design for Computer System Design: Vol 284. Vienna: Springer Vienna, 1984: 49-106.

- [11] Sweeney Paul E, Haessler Robert W. One-dimensional cutting stock decisions for rolls with multiple quality grades[J]. *European Journal of Operational Research*, 1990, 44(2): 224-231.
- [12] Schwerin Petra, Wäscher Gerhard. The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP[J]. *International Transactions in Operational Research*, 1997, 4(5): 377-389.
- [13] 巩梨, 王文璨, 刘林忠. 多目标一维装箱问题模型算法研究[J]. *计算机应用研究*, 2020, 37(S2): 144-146.
- [14] Jakobs Stefan. On genetic algorithms for the packing of polygons[J]. *European Journal of Operational Research*, 1996, 88(1): 165-181.
- [15] Hopper E, Turton B C H. An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem[J]. *European Journal of Operational Research*, 2001, 128(1): 34-57.
- [16] Martello Silvano, Monaci Michele, Vigo Daniele. An Exact Approach to the Strip-Packing Problem[J]. *INFORMS Journal on Computing*, 2003, 15(3): 310-319.
- [17] Kunpeng Li, Hailan Liu, Yong Wu, Xianhao Xu. A two-dimensional bin-packing problem with conflict penalties[J]. *International Journal of Production Research*, 2014, 52(24), 7223–7238.
- [18] Lodi Andrea, Monaci Michele, Pietroboni Enrico. Partial enumeration algorithms for Two-Dimensional Bin Packing Problem with guillotine constraints[J]. *Discrete Applied Mathematics*, 2017, 217: 40-47.
- [19] Laurent Arnaud, Klement Nathalie. Bin Packing Problem with priorities and incompatibilities using PSO: application in a health care community[J]. *IFAC-PapersOnLine*, 2019, 52(13): 2596-2601.
- [20] Zhang De Fu, Peng Yu, Zhang Li Li. A Multi-Layer Heuristic Search Algorithm for Three Dimensional Container Loading Problem[J]. *Chinese Journal of Computers*, 2012, 35(12): 2553.
- [21] Scheithauer Guntram. A Three-dimensional Bin Packing Algorithm.[J]. *Elektronische Informationsverarbeitung und Kybernetik*, 1991, 27: 263-271.

- [22] Lodi Andrea, Martello Silvano, Vigo Daniele. Heuristic algorithms for the three-dimensional bin packing problem[J]. European Journal of Operational Research, 2002, 141(2): 410-420.
- [23] Oluf Faroe, David Pisinger, Martin Zachariasen. Guided Local Search for the Three-Dimensional Bin-Packing Problem[J]. INFORMS Journal on Computing, 2003, 15(3):267-283.
- [24] Crainic Teodor Gabriel, Perboli Guido, Tadei Roberto. Extreme Point-Based Heuristics for Three-Dimensional Bin Packing[J]. INFORMS Journal on Computing, 2008, 20(3): 368-384.
- [25] Wu Yong, Li Wenkai, Goh Mark, De Souza Robert. Three-dimensional bin packing problem with variable bin height[J]. European Journal of Operational Research, 2010, 202(2): 347-355.
- [26] Kang Kyungdaw, Moon Ilkyeong, Wang Hongfeng. A hybrid genetic algorithm with a new packing strategy for the three-dimensional bin packing problem[J]. Applied Mathematics and Computation, 2012, 219(3): 1287-1299.
- [27] 吴自立, 戚铭尧. 箱型设计及装箱问题的算法研究[D].清华大学, 2018.
- [28] Gzara Fatma, Elhedhli Samir, Yildiz Burak C. The Pallet Loading Problem: Three-dimensional bin packing with practical constraints[J]. European Journal of Operational Research, 2020, 287(3): 1062-1074.
- [29] Fleszar Krzysztof. A new MILP model and fast heuristics for the variable-sized bin packing problem with time windows[J]. Computers & Industrial Engineering, 2023, 175: 108849.
- [30] Tian Ran, Kang Chunming, Bi Jiaming, Ma Zhongyu, Liu Yanxing, Yang Saisai, Li Fangfang. Learning to multi-vehicle cooperative bin packing problem via sequence-to-sequence policy network with deep reinforcement learning model[J]. Computers & Industrial Engineering, 2023, 177: 108998.
- [31] Valério De Carvalho J M. Exact solution of bin-packing problems using column generation and branch-and-bound[J]. Annals of Operations Research, 1999, 86(0): 629-659.

-
- [32] Albert E. Fernandes Muritiba, Manuel Iori, Enrico Malaguti, Paolo Toth . Algorithms for the Bin Packing Problem with Conflicts[J]. INFORMS Journal on Computing, 2008, 22(3):401-415.
- [33] Vanderbeck François. Computational study of a column generation algorithm for bin packing and cutting stock problems[J]. Mathematical Programming, 1999, 86(3): 565-594.
- [34] C.S. Chen, S.M. Lee, Q.S. Shen. An analytical model for the container loading problem[J]. European Journal of Operational Research, 1995, 80(1): 68-76.
- [35] Leonardo Junqueira, Reinaldo Morabito, Denise Sato Yamashita. Three-dimensional container loading models with cargo stability and load bearing constraints[J]. Computers & Operations Research, 2012, 39(1):74-85.
- [36] Hifi Mhand, Kacem Imed, Nègre Stéphane, Wu Lei. A Linear Programming Approach for the Three-Dimensional Bin-Packing Problem[J]. Electronic Notes in Discrete Mathematics, 2010, 36: 993-1000.
- [37] Sara Ceschia, Andrea Schaerf. Local search for a multi-drop multi-container loading problem[J]. Journal of Heuristics, 2013, 19: 275-294.
- [38] Lodi Andrea, Martello Silvano, Vigo Daniele. Heuristic algorithms for the three-dimensional bin packing problem[J]. European Journal of Operational Research, 2002, 141(2): 410-420.
- [39] Jonas Olsson, Torbjörn Larsson, Nils-Hassan Quttineh. Automating the planning of container loading for Atlas Copco: Coping with real-life stacking and stability constraints[J]. European Journal of Operational Research, 2020, 280(3): 1018-1034.
- [40] Dereli Türkay, Sena Das Gülesin. A Hybrid Simulated Annealing Algorithm for Solving Multi-Objective Container-Loading Problems[J]. Applied Artificial Intelligence, 2010, 24(5): 463-486.
- [41] Sheng Liu, Xiuqin Shang, Changjian Cheng, Hongxia Zhao, Dayong Shen, Feiyue Wang. Heuristic algorithm for the container loading problem with multiple constraints[J]. Computers & Industrial Engineering, 2017, 108: 149-164.

- [42] Pisinger David. Heuristics for the container loading problem[J]. European Journal of Operational Research, 2002, 141(2): 382-392.
- [43] Ren Jidong, Tian Yajie, Sawaragi Tetsuo. A tree search method for the container loading problem with shipment priority[J]. European Journal of Operational Research, 2011, 214(3): 526-535.
- [44] Araya I, Riff M C. A beam search approach to the container loading problem[J]. Computers & Operations Research, 2014, 43: 100-107.
- [45] VCS: A new heuristic function for selecting boxes in the single container loading problem[J]. Computers & Operations Research, 2017, 82: 27-35.
- [46] Araya Ignacio, Moyano Mauricio, Sanchez Cristobal. A beam search algorithm for the biobjective container loading problem[J]. European Journal of Operational Research, 2020, 286(2): 417-431.
- [47] Xudong Liu, Haosong Wang. A Large-Scale Tobacco 3D Bin Packing Model Based on Dual-Task Learning of Group Blocks[J]. Artificial Intelligence: Second CAAI International Conference, 2022, Part II: 71-83.
- [48] Yuan Jiang, Zhiguang Cao, Jie Zhang. Solving 3D Bin Packing Problem via Multimodal Deep Reinforcement Learning[C]. Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems, Virtual Event, United Kingdom: AAMAS, 2021: 1548–155.
- [49] Johnson, David S. Near-optimal bin packing algorithms[M]. Massachusetts, United States: Massachusetts Institute of Technology, 1973.
- [50] Qiang Liu, Huibing Cheng, Tian Tian, Yongsheng Wang, Jiewu Leng, Rongli Zhao, Hao Zhang, Lijun Wei. Algorithms for the variable sized bin packing problem[J]. Computers & Industrial Engineering, 2021, 155: 107175.

附录

装箱代码

```
# -*- coding: utf-8 -*-

import os

import sys

import csv

# 动态添加项目根目录到 sys.path

base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

sys.path.append(base_dir)

from algo.pack_tool import *

from dao.data_process import *

from utils.util import *

from datetime import datetime

import time


def main(file_path: str, log_path: str, output_path=None, result_txt=None,
detail_txt=None, summary_csv=None) -> None:

    logger = setup_logger(log_path)

    logger.info('Loading data...')

    instance_list = load_data_from_csv(file_path)

    total = len(instance_list)

    # 先写表头

    with open(result_txt, 'w', newline="", encoding='utf-8') as file:

        file.write('算例,装载率,计算时长,箱子数量\n')

    with open(detail_txt, 'w', encoding='utf-8') as detail_file, \

        open(summary_csv, 'w', newline="", encoding='utf-8') as summary_file:

        summary_writer = csv.writer(summary_file)

        summary_writer.writerow(['编号', '装载率', '计算时长', '箱子数量'])
```

```

for idx, instance in enumerate(instance_list):
    pack_tool = PackTool(instance.params, logger)
    start_time = time.perf_counter()
    best_state = pack_tool.main_algo(instance)
    end_time = time.perf_counter()
    execution_time = end_time - start_time  # 计算执行时间
    load_rate = best_state.calculate_final_load_rate(instance.params)
    container_num = getattr(best_state, 'container_num', 1)  # 没有就默
    认 1

    # 立即写入 result-summary.txt，增加箱子数量
    line = f'{instance.name},{round(load_rate,
2)},{execution_time:.6f},{container_num}\n'

    with open(result_txt, 'a', encoding='utf-8') as file:
        file.write(line)

    output_file = rf'0513-250\{instance.name}-{round(load_rate, 2)}.csv'
    output_dir = os.path.dirname(output_file)
    if output_dir and not os.path.exists(output_dir):
        os.makedirs(output_dir, exist_ok=True)
    write_solution_to_csv(output_file, best_state)
    # 写详细进度
    detail_file.write(f'算例 {idx+1}\n')
    detail_file.write(f'  进度: {idx+1}/{total}\n')
    detail_file.write(f'  装载率: {round(load_rate*100, 2)}%\n')
    detail_file.write(f'  箱子数: {container_num}\n')
    detail_file.write(f'  用时: {round(execution_time, 2)}秒\n\n')
    # 写入 summary csv
    summary_writer.writerow([idx+1, round(load_rate, 4),
round(execution_time, 4), container_num])

    return

```

```
if __name__ == '__main__':

    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) # Get
project root

    output_root = os.path.join(base_dir, '270') # 新的输出文件夹
    file_path = os.path.join(base_dir, 'test270.csv') # Assumes test.csv in project
root

    output_dir = os.path.join(base_dir, 'output')
    log_dir = os.path.join(output_dir, 'log')
    result_txt = os.path.join(base_dir, '270result.txt') # 修改为 result-summary.txt
    output_path = os.path.join(base_dir, '270result.csv') # Example
    detail_txt = os.path.join(base_dir, 'detail_result-270.txt')
    summary_csv = os.path.join(base_dir, 'summary_result-270.csv')

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    if not os.path.exists(log_dir):
        os.makedirs(log_dir) # Create the log directory

    current_time = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    log_path = os.path.join(log_dir, f'log_{current_time}.log')

    # 3. Pass correct path into your main function (and anywhere else!)
    # Assuming you have a main function defined elsewhere:
    try:
        logger = setup_logger(log_path) # setup logger
        main(file_path, log_path, output_path, result_txt, detail_txt, summary_csv)
```

```

except Exception as e:
    if logger:
        logger.exception("An error occurred: %s", e)
    else:
        print(f"An error occurred: {e}")
        positions[i].y + block.w <= space.location.y + space.shape.w
+ space.right:
        # 检查装载稳定性是否合适
        if self.check_direct_supportable(block, positions[i], loaded_blocks)
and self.check_indirect_supportable(block, positions[i], spaces):
            return True, Location(positions[i].x, positions[i].y,
positions[i].z)
        return False, Location(-1, -1, -1)

def check_direct_supportable(self, block: Block, location: Location,
loaded_blocks: list) -> bool:
    """
    Check the direct support rate of the block.(whether the boxes below the block
can support the block)
    """
    if location.z == 0:
        return True
    touch_area = 0.0
    for loaded_block in loaded_blocks:
        x1 = max(location.x, loaded_block.location.x)
        x2 = min(location.x + block.l, loaded_block.location.x + loaded_block.l)
        y1 = max(location.y, loaded_block.location.y)
        y2 = min(location.y + block.w, loaded_block.location.y +
loaded_block.w)

```

```

        z1 = max(location.z, loaded_block.location.z)
        z2 = min(location.z + block.h, loaded_block.location.z +
loaded_block.h)
        # 如果在三个投影面都有图形重叠，则返回 false
        if x1 < x2 and y1 < y2 and z1 < z2:
            return False
        if 0 <= location.z - (loaded_block.location.z + loaded_block.h) <=
self.params.height_diff_in_one_layer:
            touch_area += max(x2 - x1, 0) * max(y2 - y1, 0)
        return touch_area >= self.params.min_support_area_rate * block.l * block.w

def check_indirect_supportable(self, block: Block, location: Location, spaces: list)
-> bool:
    """
    Check the indirect support rate of the block.(whether the free spaces below
the block satisfies certain conditions)
    """
    if location.z == 0:
        return True
    projected_area = 0.0
    for space in spaces:
        # 只考虑纵坐标小于 block 装载位置纵坐标的空白区域
        if space.location.z < location.z - self.params.height_diff_in_one_layer:
            x1 = max(location.x, space.location.x - space.back)
            x2 = min(location.x + block.l, space.location.x + space.shape.l +
space.front)
            y1 = max(location.y, space.location.y - space.left)
            y2 = min(location.y + block.w, space.location.y + space.shape.w +
space.right)

```

```

        projected_area = max(max(x2 - x1, 0) * max(y2 - y1, 0),
projected_area) # 取最大的投影面积

        return projected_area <= self.params.max_projected_area_rate * block.l *
block.w

# ===== 子空间更新函数 =====
def update_subspaces(self, spaces: list, block: Block, location: Location) -> None:
    """
    Update subspaces after loading a block
    """
    new_spaces = []
    can_removable_spaces = []
    # split spaces
    for space in spaces:
        x1 = min(space.location.x + space.shape.l + space.front, location.x +
block.l)
        x2 = max(space.location.x - space.back, location.x)
        y1 = min(space.location.y + space.shape.w + space.right, location.y +
block.w)
        y2 = max(space.location.y - space.left, location.y)
        z1 = min(space.location.z + space.shape.h, location.z + block.h)
        z2 = max(space.location.z, location.z)
        # check if the free space overlaps with block
        if x1 > x2 and y1 > y2 and z1 > z2:
            can_removable_spaces.append(space)
        # 处理前面的空闲子空间
        if space.location.x + space.shape.l > location.x + block.l +
self.params.gap:

```

```

        front = Space(Location(location.x + block.l + self.params.gap,
space.location.y, space.location.z),
                        Shape(space.location.x + space.shape.l -
(location.x + block.l + self.params.gap), space.shape.w, space.shape.h),
                        space.front, 0, space.left, space.right)
        front.set_anchor(self.params)
        new_spaces.append(front)
    # 处理后面的空闲子空间
    if location.x - self.params.gap > space.location.x:
        back = Space(space.location, Shape(location.x -
space.location.x - self.params.gap, space.shape.w, space.shape.h),
                    0, space.back, space.left, space.right)
        back.set_anchor(self.params)
        new_spaces.append(back)
    # 处理左边的空闲子空间
    if space.location.y < location.y - self.params.gap:
        left = Space(space.location, Shape(space.shape.l, location.y -
space.location.y - self.params.gap, space.shape.h),
                    space.front, space.back, space.left, 0)
        left.set_anchor(self.params)
        new_spaces.append(left)
    # 处理右边的空闲子空间
    if space.location.y + space.shape.w - (location.y + block.w +
self.params.gap) > 0:
        right = Space(Location(space.location.x, location.y + block.w
+ self.params.gap, space.location.z),
                        Shape(space.shape.l, space.location.y +
space.shape.w - (location.y + block.w + self.params.gap), space.shape.h),
                        space.front, space.back, 0, space.right)

```

```

        right.set_anchor(self.params)
        new_spaces.append(right)
    # 处理下方空闲子空间
    if location.z - space.location.z > 0:
        below = Space(space.location, Shape(space.shape.l,
space.shape.w, location.z - space.location.z),
                        space.front, space.back, space.left,
space.right)

        below.set_anchor(self.params)
        new_spaces.append(below)
    # 处理上方空闲子空间
    if space.location.z + space.shape.h - (location.z + block.h) > 0:
        above = Space(
            Location(max(space.location.x - space.back, location.x),
max(space.location.y - space.left, location.y),
                    location.z + block.h),
            Shape(min(block.l, location.x + block.l -
(space.location.x - space.back)),
                    min(block.w, location.y + block.w -
(space.location.y - space.left)),
                    space.location.z + space.shape.h - (location.z +
block.h)),
            max(0, space.location.x + space.shape.l + space.front -
location.x - block.l),
            max(0, location.x - (space.location.x - space.back)),
            max(0, location.y - (space.location.y - space.left)),
            max(0, space.location.y + space.shape.w + space.right -
location.y - block.w)
        )

```

```

        above.set_anchor(self.params)

        new_spaces.append(above)

# remove the spaces that can be split
if len(can_removable_spaces) > 0:
    for space in can_removable_spaces:
        spaces.remove(space)

    can_removable_spaces.clear()

# merge spaces: 首先增加新空间 之后进行合并操作
spaces.extend(new_spaces)

spaces.sort(key=lambda x: x.volume, reverse=True)

for i in range(len(spaces) - 1):
    if spaces[i] in can_removable_spaces:
        continue

    for j in range(i + 1, len(spaces)):
        if spaces[j] in can_removable_spaces:
            continue

        if spaces[i].location.x <= spaces[j].location.x and
spaces[i].location.y <= spaces[j].location.y and \
            spaces[i].location.z <= spaces[j].location.z:
            if spaces[i].location.x + spaces[i].shape.l >=
spaces[j].location.x + spaces[j].shape.l and \
                spaces[i].location.y + spaces[i].shape.w >=
spaces[j].location.y + spaces[j].shape.w and \
                    spaces[i].location.z + spaces[i].shape.h >=
spaces[j].location.z + spaces[j].shape.h:
                if spaces[i].front >= spaces[j].front and
spaces[i].back >= spaces[j].back and spaces[i].left >= spaces[j].left and \
                    spaces[i].right >= spaces[j].right:
                        can_removable_spaces.append(spaces[j])

```

```

        if len(can_removable_spaces) > 0:
            for space in can_removable_spaces:
                spaces.remove(space)

    return

# ===== block 生成
函数 =====
def generate_blocks(self, instance: Instance) -> tuple[dict, list]:
    """
    Generate blocks by combining simple blocks and complex blocks and group
    which by height.
    """
    block_dict = dict()
    heights = []
    # generate simple blocks
    simple_blocks = self.generate_simple_blocks(instance)
    # add simple blocks to block dict
    block_id = 0
    for block in simple_blocks:
        if block.h not in block_dict:
            block_dict[block.h] = []
            heights.append(block.h)
        if block not in block_dict[block.h]:
            block_id += 1
            block.id = block_id
            block_dict[block.h].append(block)
    # generate general blocks
    general_blocks = self.generate_general_blocks(simple_blocks)
    # add general blocks to block dict

```

```
for block in general_blocks:
    if block.h not in block_dict:
        block_dict[block.h] = []
        heights.append(block.h)
    if block not in block_dict[block.h]:
        block_id += 1
        block.id = block_id
        block_dict[block.h].append(block)
return block_dict, heights
```

```
def generate_general_blocks(self, blocks: list) -> list:
    """
    Generate general blocks by combining simple blocks
    """
    B = blocks
    PG = []
    PG.extend(B)
    while len(B) < self.params.max_block_num and len(PG) > 0:
        NG = []
        res_num = self.params.max_block_num - len(B)
        for b1 in PG:
            if res_num == 0:
                break
            for b2 in B:
                if res_num == 0:
                    break
                if self.is_crossed_blocks(b1, b2):
                    continue
                if b1.h == b2.h:
```

```

# 1. l-direction composite
if b1.volume + b2.volume >=
self.params.min_composite_flr * max(b1.w, b2.w) * (b1.l + b2.l) * b1.h and \
    b1.l + b2.l <= self.params.max_length:
    b_l = self.combine_blocks_with_dimension(b1, b2,
'l')

    if b_l not in NG and b_l not in B:
        NG.append(b_l)
        res_num -= 1

# 2. w-direction composite
if b1.volume + b2.volume >=
self.params.min_composite_flr * max(b1.l, b2.l) * (b1.w + b2.w) * b1.h and \
    b1.w + b2.w <= self.params.max_load_width:
    b_w = self.combine_blocks_with_dimension(b1,
b2, 'w')

    if b_w not in NG and b_w not in B:
        NG.append(b_w)
        res_num -= 1

# 3. h-direction composite
if b1.volume + b2.volume >= self.params.min_composite_flr
* max(b1.l, b2.l) * max(b1.w, b2.w) * (b1.h + b2.h) and \
    b1.h + b2.h <= self.params.max_load_height:
    b_h = self.combine_blocks_with_dimension(b1, b2, 'h')
    if b_h not in NG and b_h not in B:
        NG.append(b_h)
        res_num -= 1

PG = NG

for block in NG:
    if len(B) >= self.params.max_block_num:

```

```
        break
    B.append(block)
return B

def generate_simple_blocks(self, instance: Instance) -> list:
    """
    Generate simple blocks from the given single box
    """
    blocks = []
    for box in instance.bboxes:
        # 1.原方向
        b1 = Block(l=box.origin_l, w=box.origin_w, h=box.origin_h)
        b1.bboxes.append(copy.deepcopy(box))
        b1.volume = box.volume
        b1.relative_location.append(Location(0, 0, 0))
        blocks.append(b1)
        # 2.旋转 90 度
        if box.l <= self.params.max_load_width:
            b2 = Block(l=box.origin_w, w=box.origin_l, h=box.h)
            box_copy = copy.deepcopy(box)
            box_copy.l = box.origin_w
            box_copy.w = box.origin_l
            box_copy.direction = 90
            b2.bboxes.append(box_copy)
            b2.volume = box_copy.volume
            b2.relative_location.append(Location(0, 0, 0))
            blocks.append(b2)
    return blocks
```

```

def combine_blocks_with_dimension(self, b1: Block, b2: Block, dimension: str)
-> Block:
    """
    Combine two blocks with specific dimension
    """
    if dimension == 'l':
        b = Block(l=b1.l + b2.l, w=max(b1.w, b2.w), h=b1.h)
        b.volume = b1.volume + b2.volume
        # add boxes
        if b1.w >= b2.w:
            b.bboxes.extend(b1.bboxes)
            b.bboxes.extend(b2.bboxes)
            b.relative_location.extend(b1.relative_location)
            for location in b2.relative_location:
                b.relative_location.append(Location(b1.l + location.x,
location.y, location.z))
        else:
            b.bboxes.extend(b2.bboxes)
            b.bboxes.extend(b1.bboxes)
            b.relative_location.extend(b2.relative_location)
            for location in b1.relative_location:
                b.relative_location.append(Location(b2.l + location.x,
location.y, location.z))
    elif dimension == 'w':
        b = Block(l=max(b1.l, b2.l), w=b1.w + b2.w, h=b1.h)
        b.volume = b1.volume + b2.volume
        # add boxes
        if b1.w >= b2.w:
            b.bboxes.extend(b1.bboxes)

```

```

        b.bboxes.extend(b2.bboxes)
        b.relative_location.extend(b1.relative_location)
        for location in b2.relative_location:
            b.relative_location.append(Location(location.x,    b1.w    +
location.y, location.z))
        else:
            b.bboxes.extend(b2.bboxes)
            b.bboxes.extend(b1.bboxes)
            b.relative_location.extend(b2.relative_location)
            for location in b1.relative_location:
                b.relative_location.append(Location(location.x,    b2.w    +
location.y, location.z))
        else:
            b = Block(l=max(b1.l, b2.l), w=max(b1.w, b2.w), h=b1.h + b2.h)
            b.volume = b1.volume + b2.volume
            # add boxes
            if b1.l * b1.w >= b2.l * b2.w: # 以长边作为衡量标准
                b.bboxes.extend(b1.bboxes)
                b.bboxes.extend(b2.bboxes)
                b.relative_location.extend(b1.relative_location)
                for location in b2.relative_location:
                    b.relative_location.append(Location(location.x,    location.y,
b1.h + location.z))
            else:
                b.bboxes.extend(b2.bboxes)
                b.bboxes.extend(b1.bboxes)
                b.relative_location.extend(b2.relative_location)
                for location in b1.relative_location:

```

```

        b.relative_location.append(Location(location.x, location.y,
b2.h + location.z))
    return b

```

```

def is_crossed_blocks(self, block1: Block, block2: Block) -> bool:

```

```

    """

```

```

    check if two blocks are crossed

```

```

    """

```

```

    for box1 in block1.bboxes:

```

```

        if box1 in block2.bboxes:

```

```

            return True

```

```

    return False

```

```

# 评价函数系列

```

```

def calculate_touch_area_of_block_in_space(self, block: Block, location:
Location, packing_set: PackingSet) -> float:

```

```

    """

```

得到一个 block 放入某一个空闲子空间后与已装载 box 以及箱体的接触面积

```

    """

```

```

    start = packing_set.start

```

```

    loaded_blocks = packing_set.loaded_blocks

```

```

    touch_area = 0.0

```

```

    # 计算 block 与箱体前后面的接触面积

```

```

    if location.x <= start.x + self.params.min_box_gap:

```

```

        touch_area += block.w * block.h

```

```

    if location.x + block.l >= self.params.max_load_length - start.x -
self.params.min_box_gap:

```

```

        touch_area += block.w * block.h

```

```

# 计算 block 与箱体左右面的接触面积
if location.y <= self.params.min_box_gap:
    touch_area += block.l * block.h
    if location.y + block.w >= self.params.max_load_width -
self.params.min_box_gap:
        touch_area += block.l * block.h
# 计算 block 与箱体上下面的接触面积
if location.z == 0:
    touch_area += block.l * block.w
    if location.z + block.h == self.params.max_load_height:
        touch_area += block.l * block.w
# 计算 block 与其他 block 的接触面积
point1_0 = location
point1_1 = Location(location.x + block.l, location.y + block.w, location.z +
block.h)
for loaded_block in loaded_blocks:
    point2_0 = loaded_block.location
    point2_1 = Location(loaded_block.location.x + loaded_block.l,
loaded_block.location.y + loaded_block.w,
loaded_block.location.z + loaded_block.h)
    x1 = max(point1_0.x, point2_0.x)
    x2 = min(point1_1.x, point2_1.x)
    y1 = max(point1_0.y, point2_0.y)
    y2 = min(point1_1.y, point2_1.y)
    z1 = max(point1_0.z, point2_0.z)
    z2 = min(point1_1.z, point2_1.z)

    if point2_1.y + self.params.gap == point1_0.y:
        touch_area += max(x2 - x1, 0) * max(z2 - z1, 0)

```

```

        if point2_0.y == point1_1.y + self.params.gap:
            touch_area += max(x2 - x1, 0) * max(z2 - z1, 0)
        if point2_1.x + self.params.gap == point1_0.x:
            touch_area += max(y2 - y1, 0) * max(z2 - z1, 0)
        if point2_0.x == point1_1.x + self.params.gap:
            touch_area += max(y2 - y1, 0) * max(z2 - z1, 0)
        if point1_0.z - self.params.height_diff_in_one_layer <= point2_1.z <=
point1_0.z:
            touch_area += max(x2 - x1, 0) * max(y2 - y1, 0)
        if point1_1.z <= point2_0.z <= point1_1.z +
self.params.height_diff_in_one_layer:
            touch_area += max(x2 - x1, 0) * max(y2 - y1, 0)
    return touch_area

def calculate_vcs(self, block: Block, location: Location, packing_set: PackingSet)
-> float:
    """
    Calculate the vcs value after placing a block into space.
    """
    touch_area = self.calculate_touch_area_of_block_in_space(block, location,
packing_set)
    surface_area = 2 * (block.l * block.w + block.w * block.h + block.h * block.l)
    ran = random.random() if self.params.random_factor > 0.0 else 1.0

    return block.volume * (touch_area / surface_area) ** self.params.alpha +
self.params.random_factor * math.log(ran)

```